香港中文大學統計學系
Department of Statistics
THE CHINESE UNIVERSITY OF HONG KONG

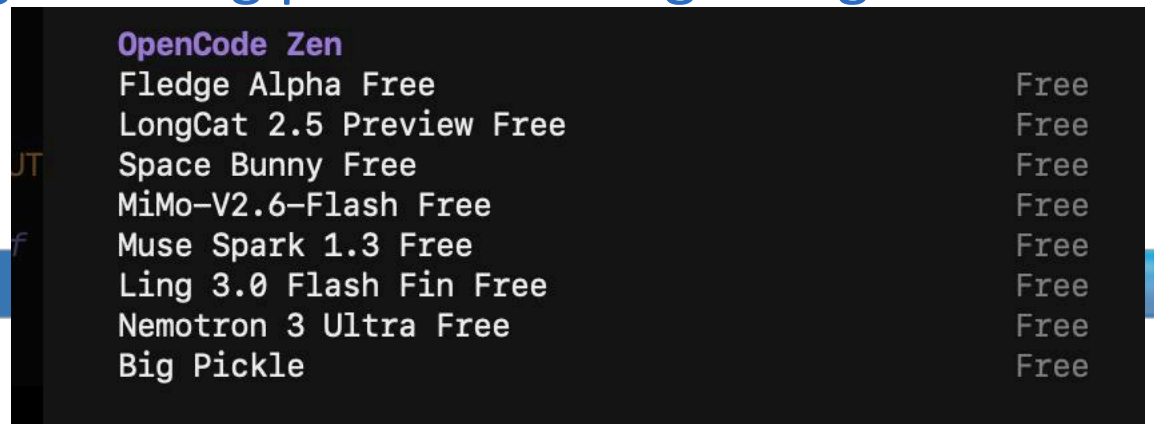
STAT6207
Applied Deep Learning

L5, 9 Oct 2026
By Dr. Ming Cheung



Midterm exam (23 Oct 26, 7pm-930pm)

- Exam room: CYT201, 202, 209B
- There will be two parts
  - Written: MC and short question
  - Programming: You can use your computer and agent coder
- No communication is allowed
- You can bring a cheat sheet (A4, double page)
- Make sure that you can run all demo and assignments on your computer
- You will need to complete the written part first
- You will know the questions in Programming part at the beginning
- You will need to use free models



Announcement

- Final Project
  - 4-5 persons in a group, no need to send me your group list now
  - The topic will be announced after mid term
- Assignment 2: prompt
  - **Generalization to Unseen Data:** Your submitted prompt will be evaluated against a test dataset that has the exact same structure (identical columns) as the dataset you received, but covers a different time period. Please design your prompt carefully to ensure it generalizes well to unseen data.
  - **Word Count Limit (Strict < 500 words):** Your prompt must not exceed 500 words. Because space is limited, focus primarily on the **deep learning components** (e.g., model selection, hyperparameter settings, normalization techniques) rather than general system setup instructions (e.g., library imports).
  - Please submit your prompt as “prompt.txt”
- Assignment 2: The prediction target is Appliances



Lecture Room - next lecture (L6): BMS G18

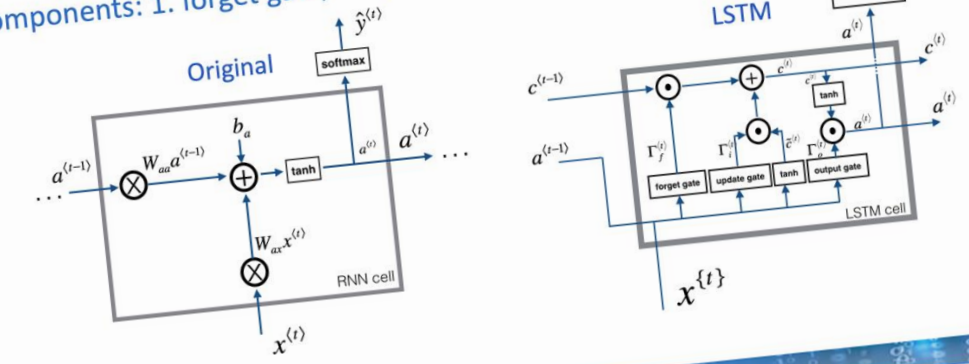
Lecture	Week	Lecture Rooms
L5	9-Oct	CYT LT1A
L6	16-Oct	BMS G18
L7 (Mid Term)	23-Oct	CYT201, 202, 209B
L8	30-Oct	CYT LT1A
L9	6-Nov	UCC C1
L10	13-Nov	CYT LT1A
L11	20-Nov	UCC C1
L12	27-Nov	UCC C1
L13	4-Dec	BMS G18
L14	11-Dec	BMS G18



Last Lecture

New: Long Short-Term Memory (LSTM)

- components: 1. forget gate, 2. update gate, 3. output gate, 4. cell state



The Softmax—Similarity to Probability

- Suppose we have a vector of scores: [2.0, 1.0, 0.5]. To apply softmax:
  - e^2.0 ≈ 7.389
  - e^1.0 ≈ 2.718
  - e^0.5 ≈ 1.649
  - Sum: 7.389 + 2.718 + 1.649 = 11.756
- Finally, the values:
  - 7.389 / 11.756 ≈ 0.628
  - 2.718 / 11.756 ≈ 0.231
  - 1.649 / 11.756 ≈ 0.140

Softmax activation function

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

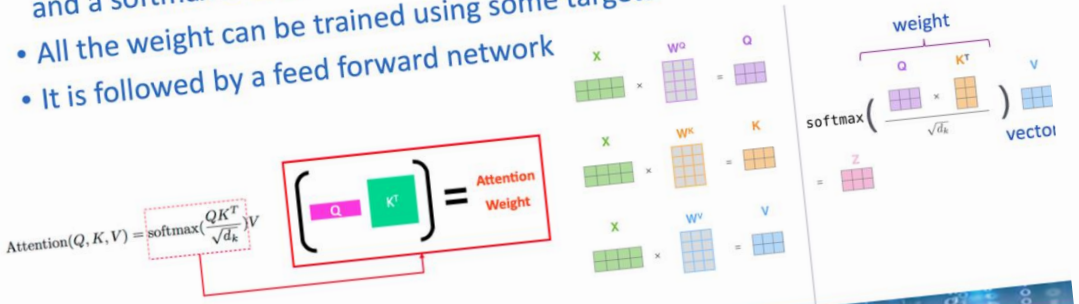
Softmax in LLM

$$\frac{e^{\frac{y_i}{T}}}{\sum_{k=1}^{n} e^{\frac{y_k}{T}}}$$

44

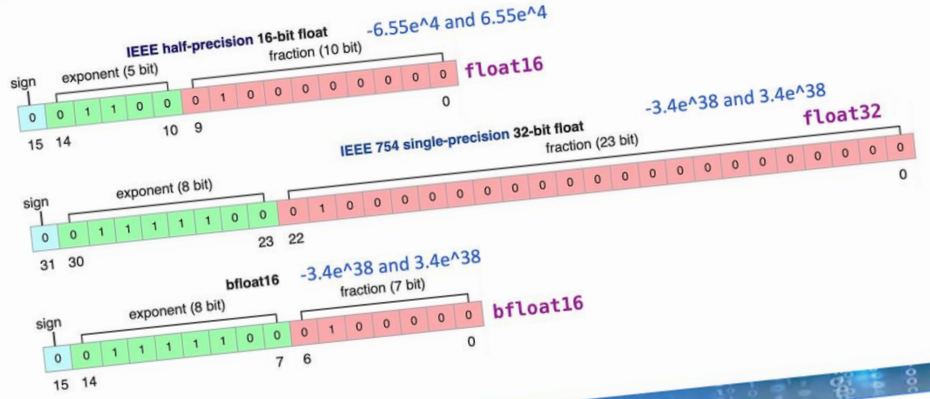
Computing attention weight

- The Q, K and V are formed using **the same input X** with different weight, and a softmax is used to compute **how much attention is put on V**.
- All the weight can be trained using some targets
- It is followed by a feed forward network



45

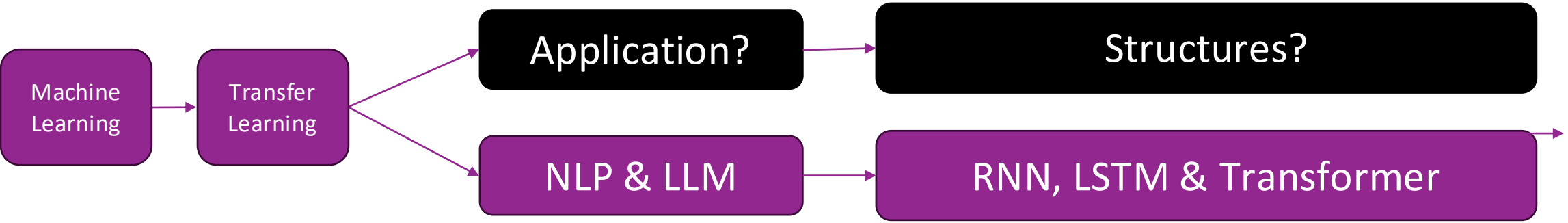
Quantization

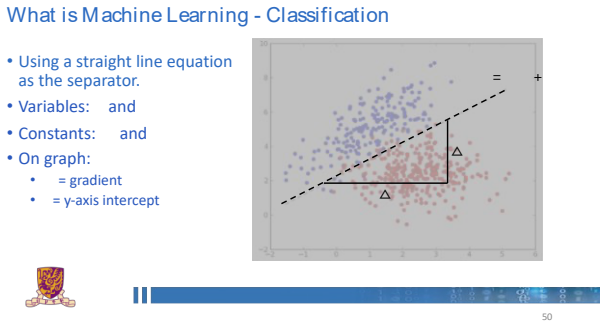


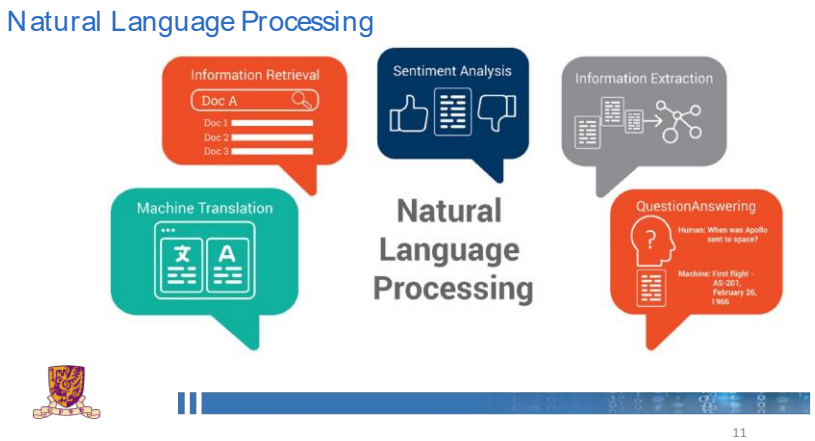
81

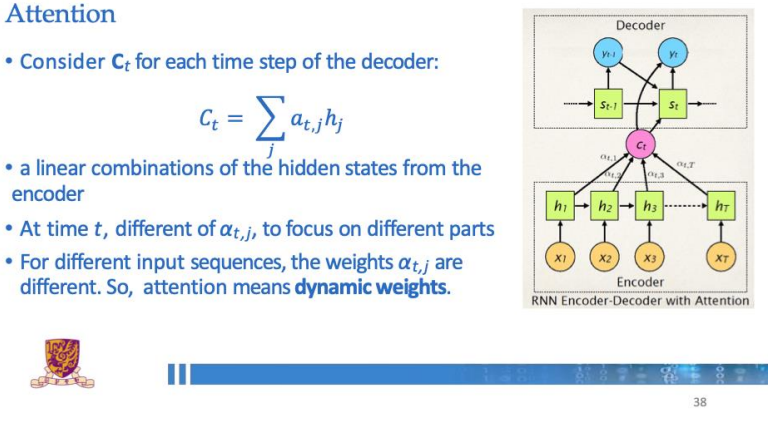
Course so far

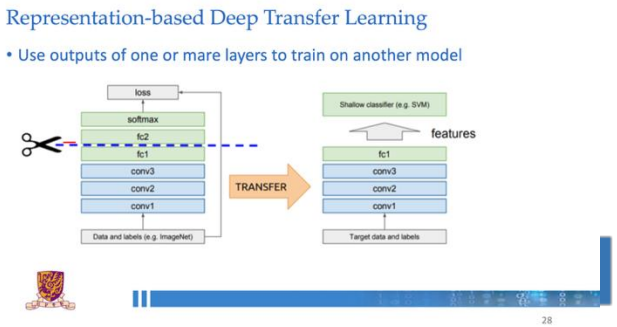
- How do we understand images?







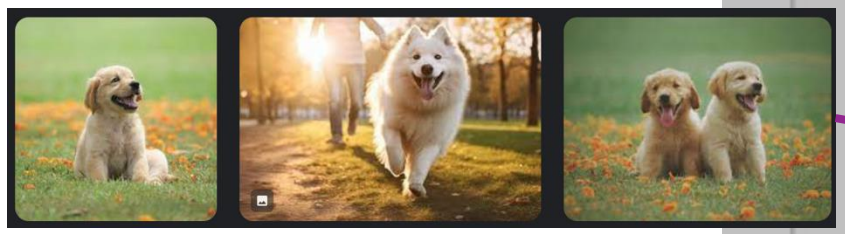


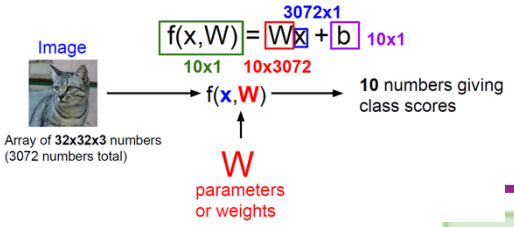


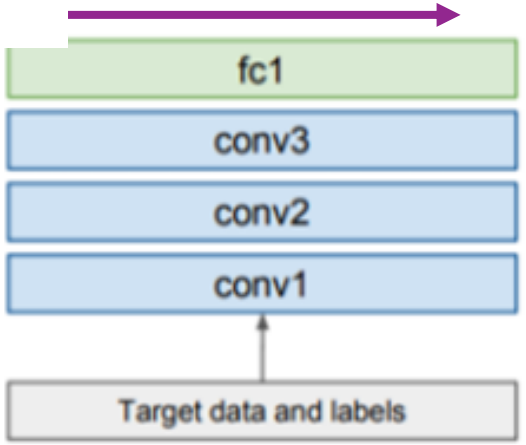
Agenda

- Computer Vision (CV) and Convolutional Neural Networks (CNN)
- Implementation of CV (OpenCV+PyTorch)
- Multi-model LLM and VLM



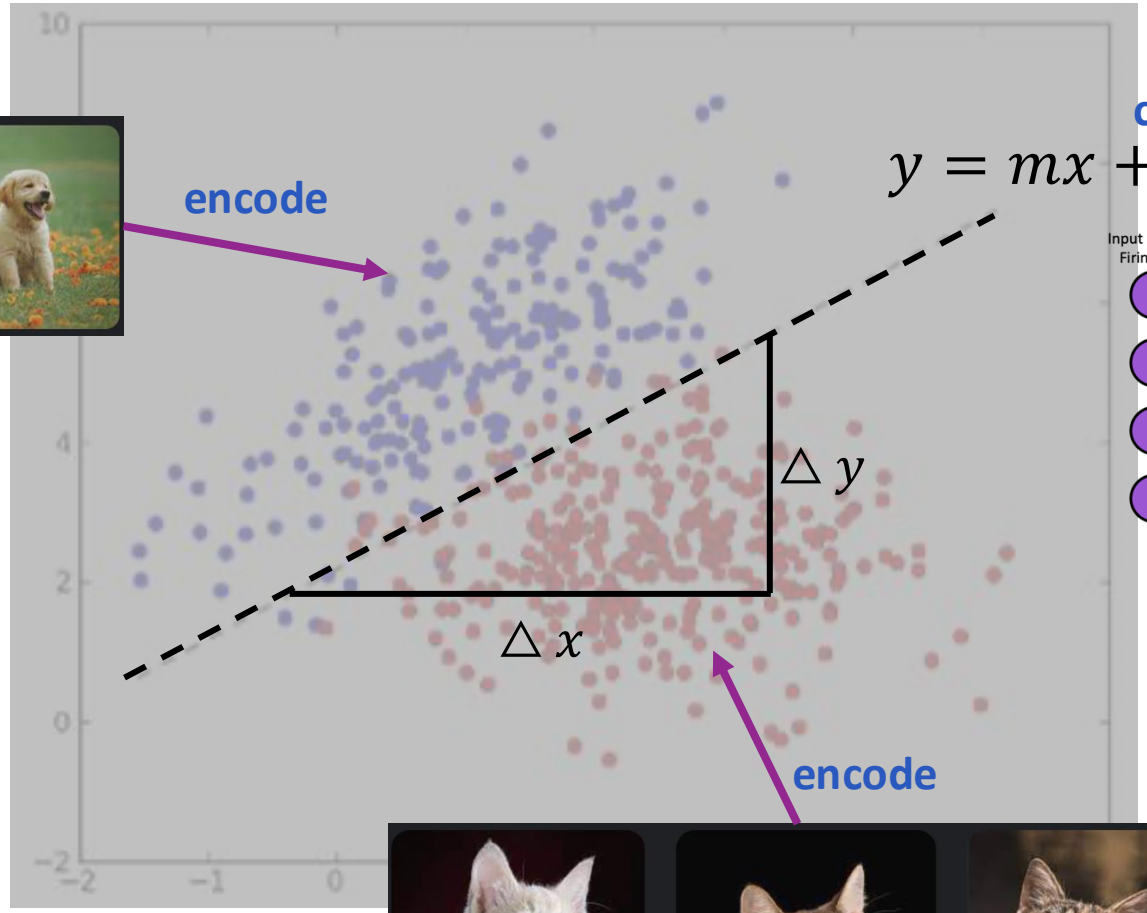




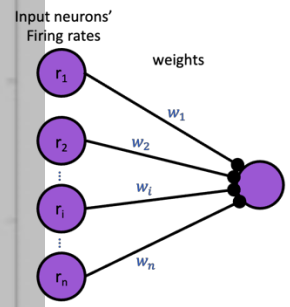


Pre-train model (encoder)

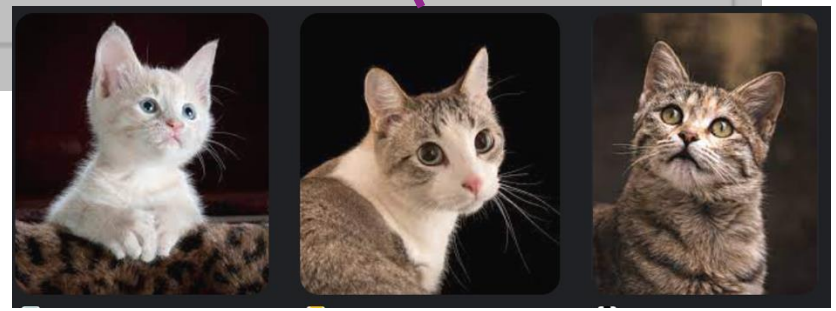
encode



classifier

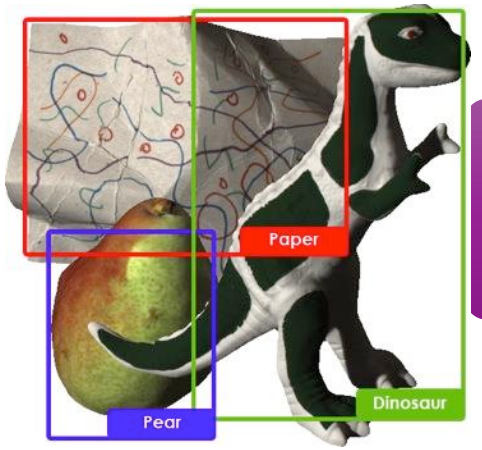


encode

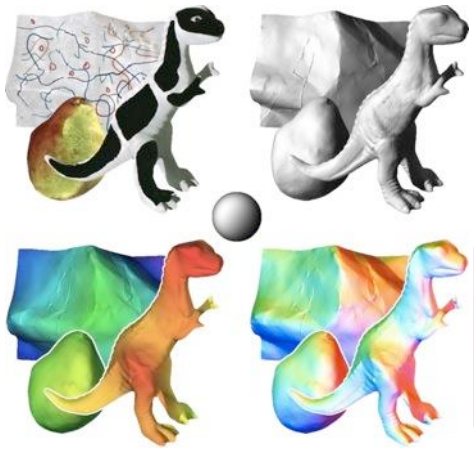




Recognition, Reconstruction & Reorganization

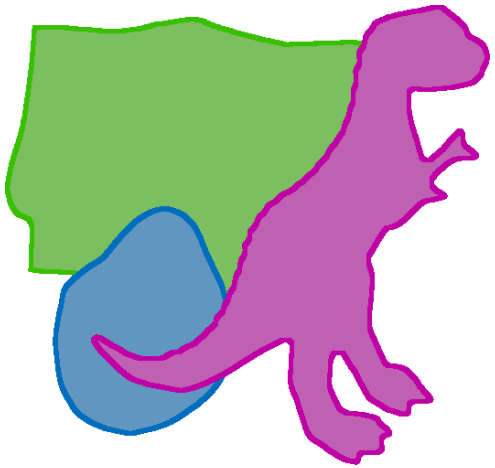


Recognition



Reconstruction

Reorganization



What CV is looking for …

- What do you see?

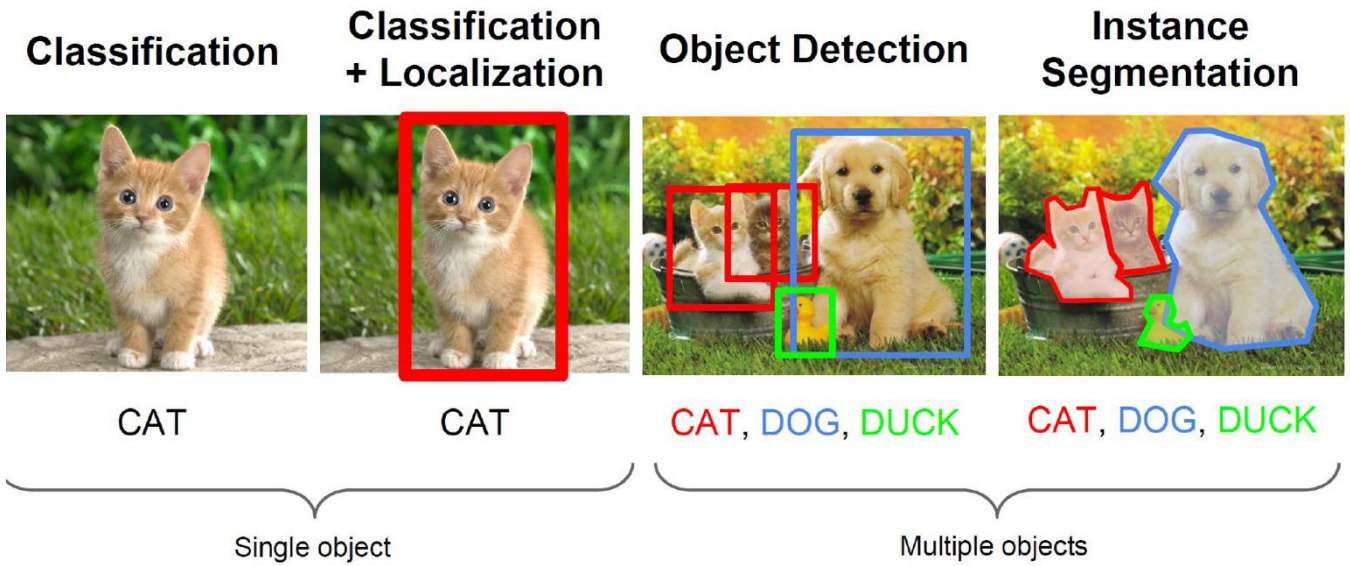


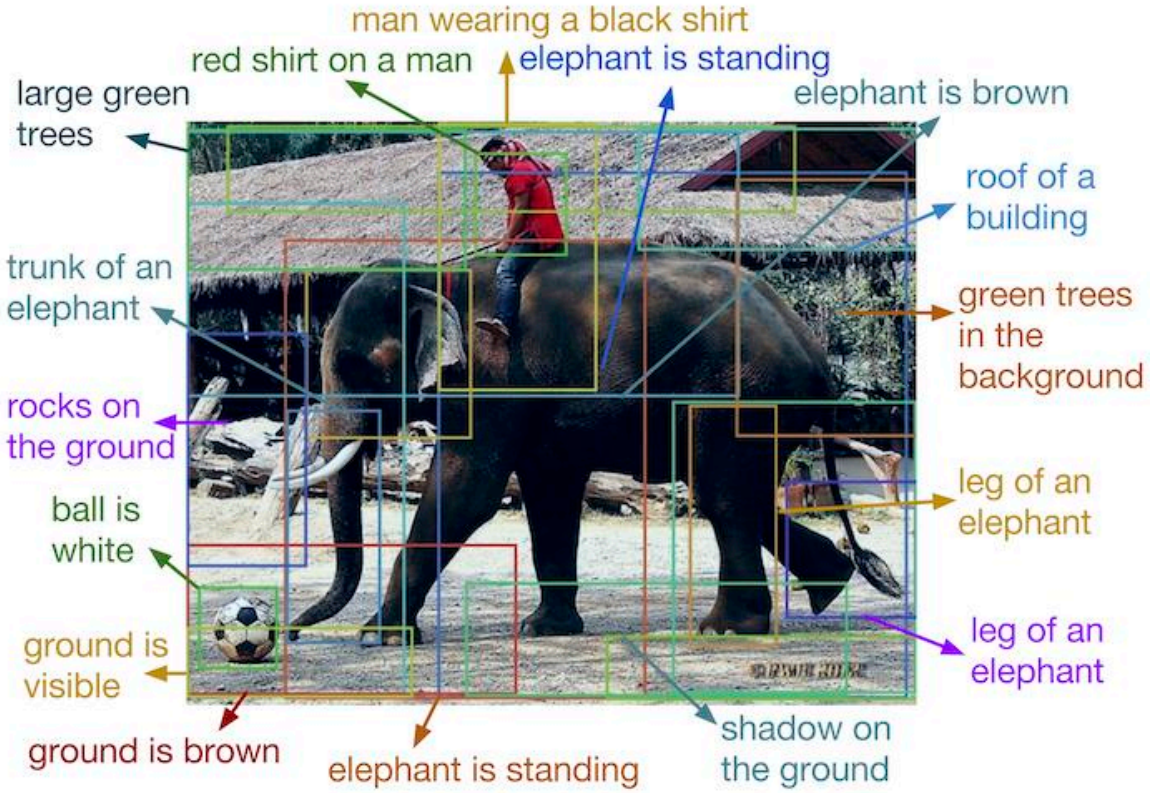
Some problems that we can solve…

- What can computer “see”



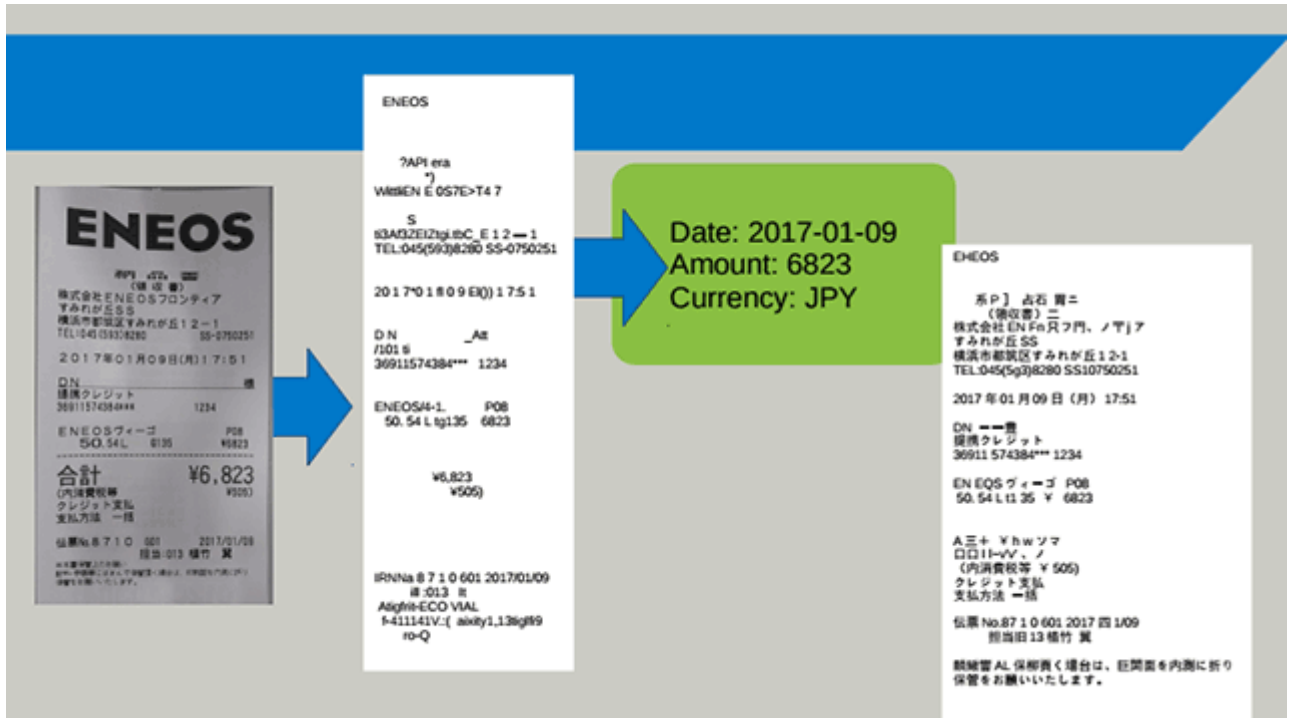
Common areas in CV





Optical Character Recognition (OCR)

- Recognize texts on an image



YCG09/chinese_ocr
CTPN + DenseNet + CTC based end-to-end Chinese OCR implemented using tenso
2.6k Python Apache-2.0 license Updated on 8 Oct 2019

faustomorales/keras-ocr
A packaged and flexible version of the CRAFT text detector and Keras CRNN recogn
ocr keras text-detection keras-crnn
1.1k Python MIT license Updated 3 days ago

xiaomaxiao/keras_ocr
用keras实现OCR定位、识别
524 Jupyter Notebook Updated on 21 Apr 2019

xiaofengShi/CHINESE-OCR
[python3.6] 运用tf实现自然场景文字检测,keras/pytorch实现ctpn+crnn+ctc实现不定长
ocr tensorflow ctpn pytorch-crnn keras-crnn lstm-ctc
2.7k Python Updated on 13 Aug 2019

qjadud1994/CRNN-Keras
CRNN (CNN+RNN) for OCR using Keras / License Plate Recognition
481 Python MIT license Updated on 26 May 2020

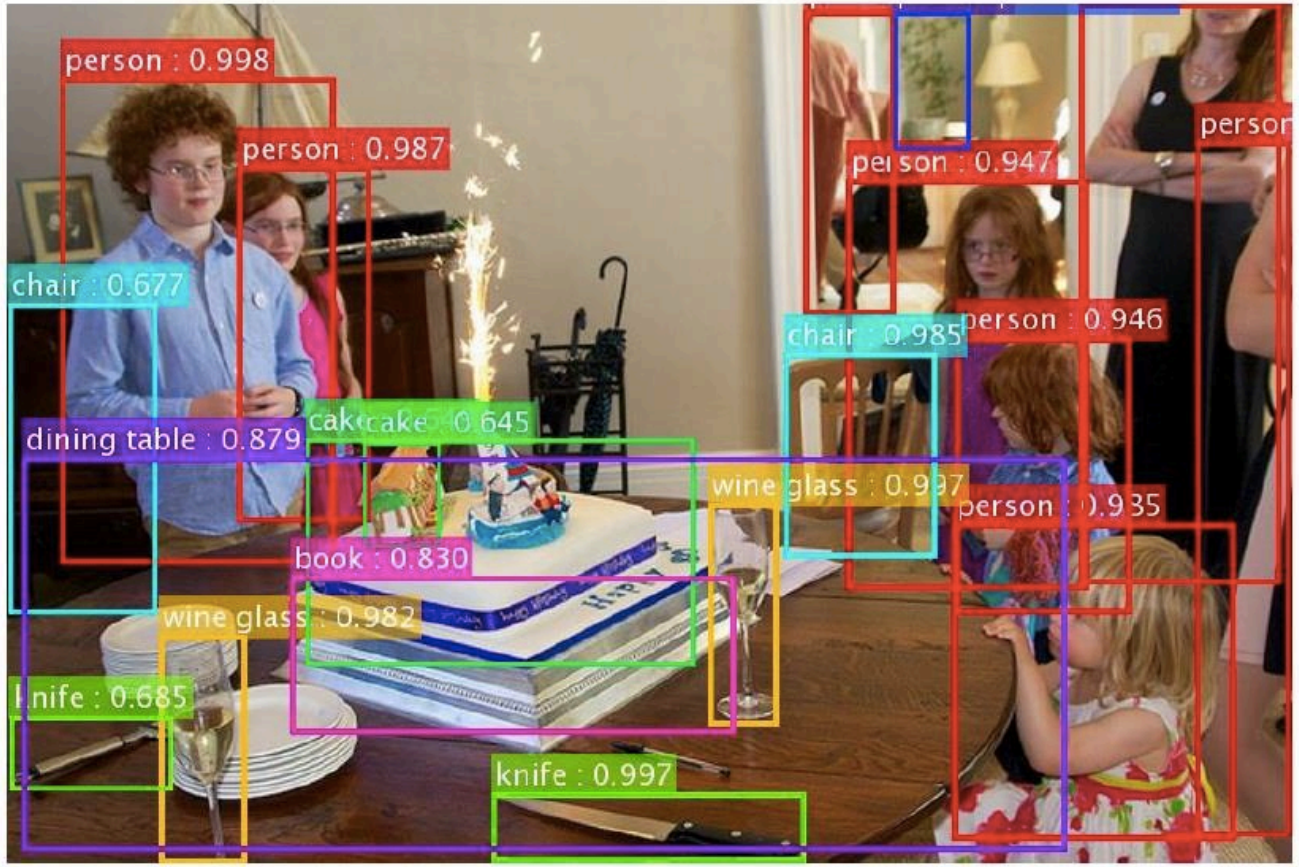
MachineLP/OCR_Keras
OCR；文本检测、文本识别(cnn+ctc、crnn+ctc)。
195 Python Updated on 22 Jun

GlassyWing/text-detection-ocr
Chinese text detection and recognition based on CTPN + DENSENET using Keras an
keras和tensorflow基于CTPN+...
274 Python Apache-2.0 license Updated on 9 Jan 2020

isee15/Card-Ocr
身份证识别OCR
ocr keras tesseract python3
407 Jupyter Notebook Updated on 22 Sep 2021

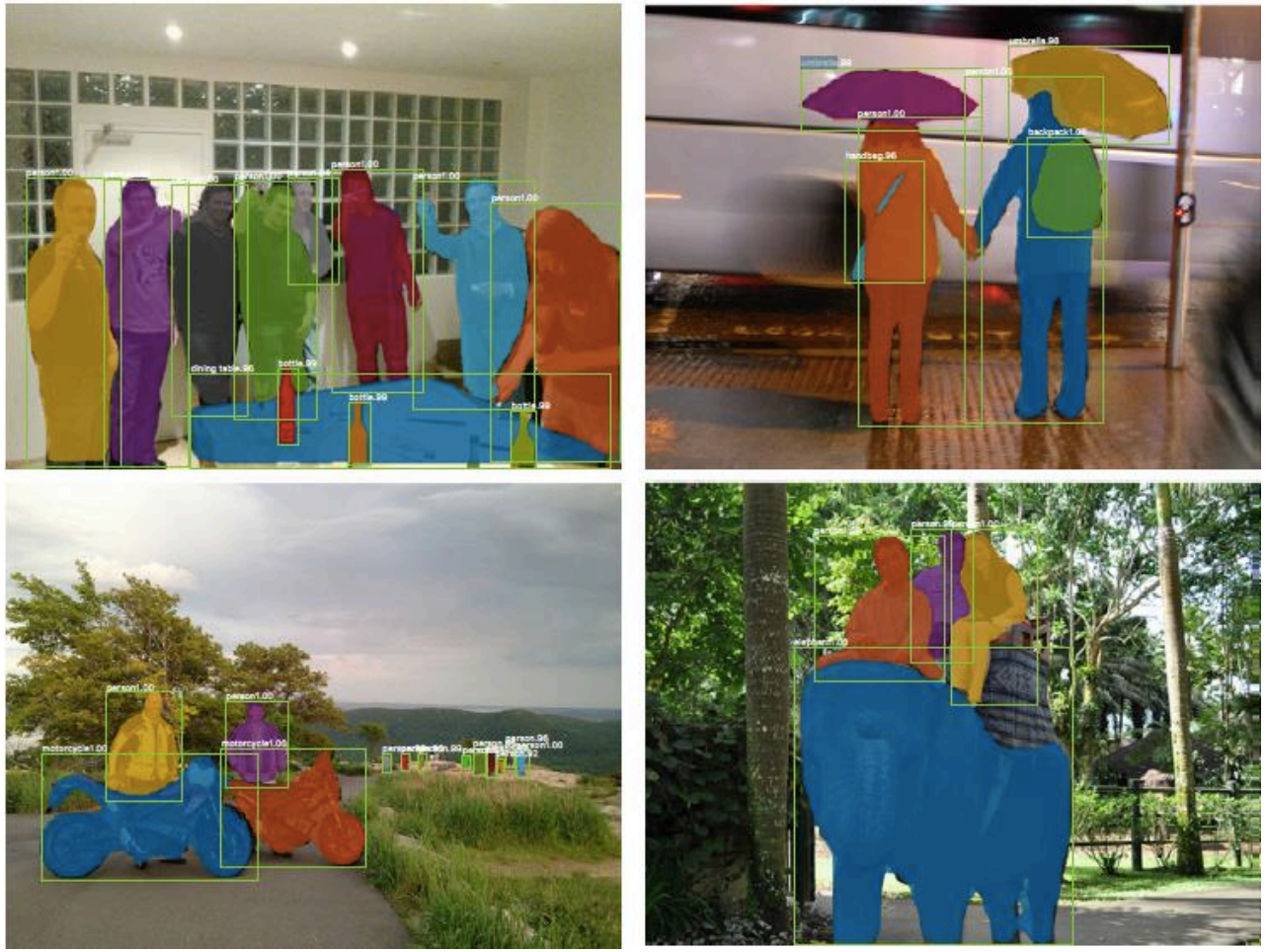
Object Detection

- Detect and recognize objects and humans from images



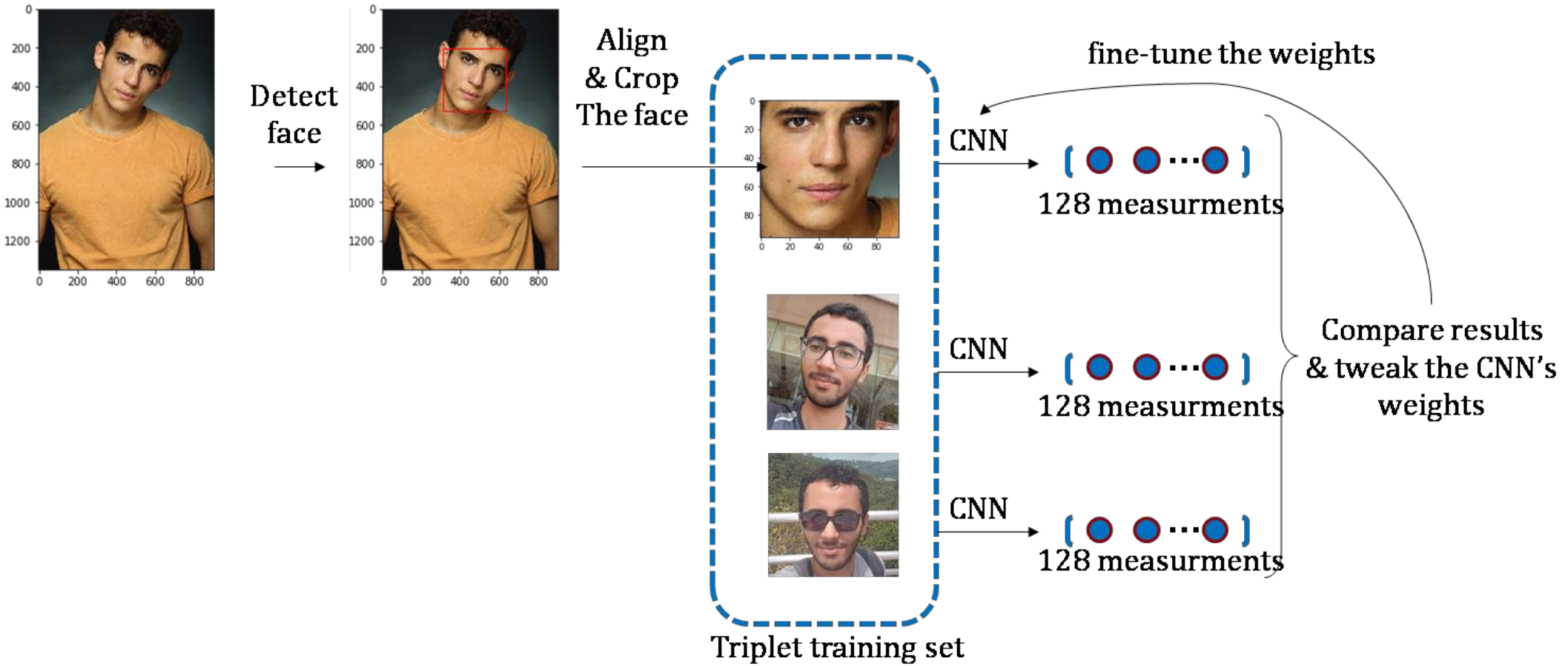
Object Segmentation

- Segment objects/humans



Face Detection & Recognition

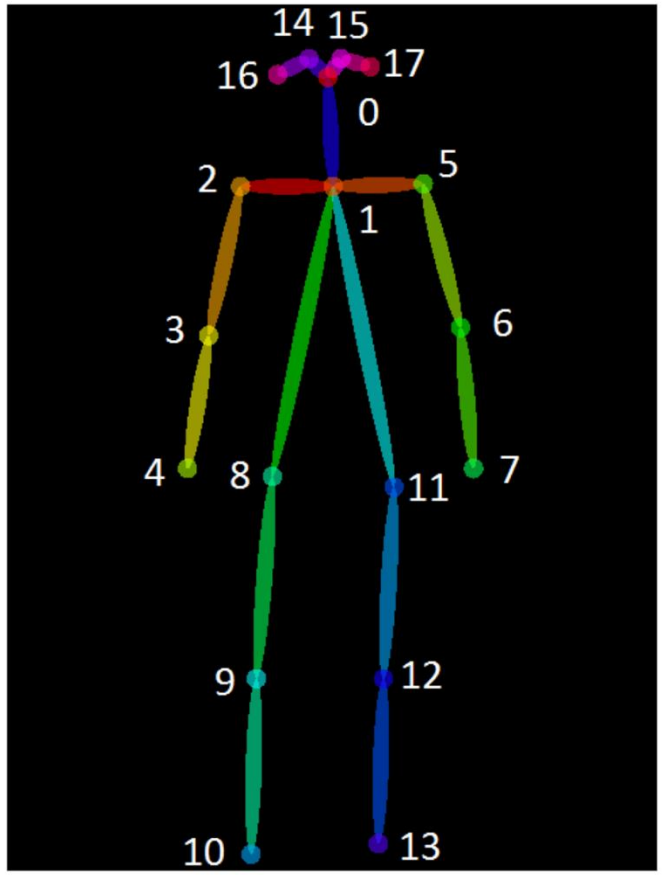
- Encode faces into vectors



Human Segmentation



Human Joints Detection

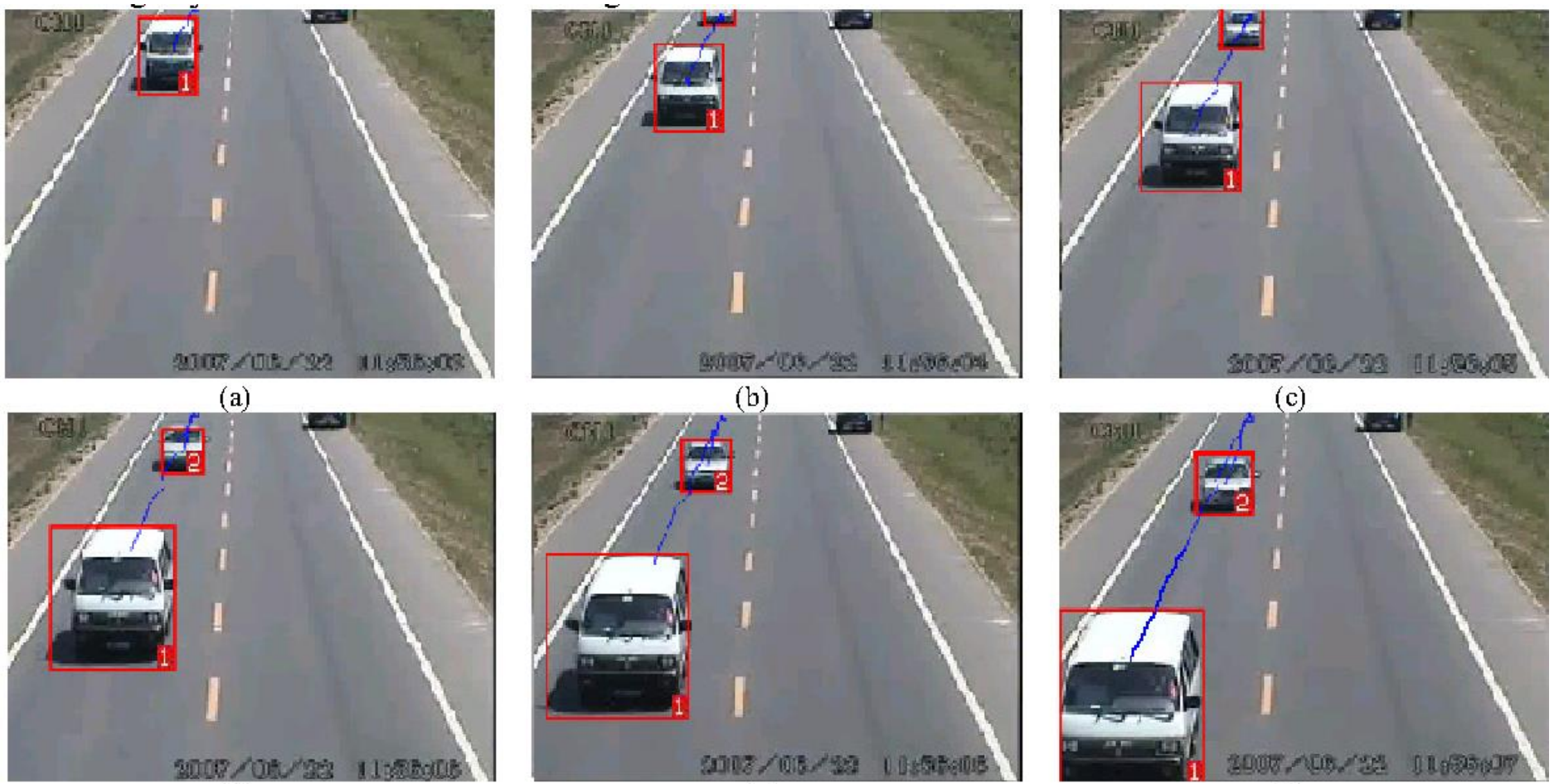




Fashion Analysis



Object Tracking



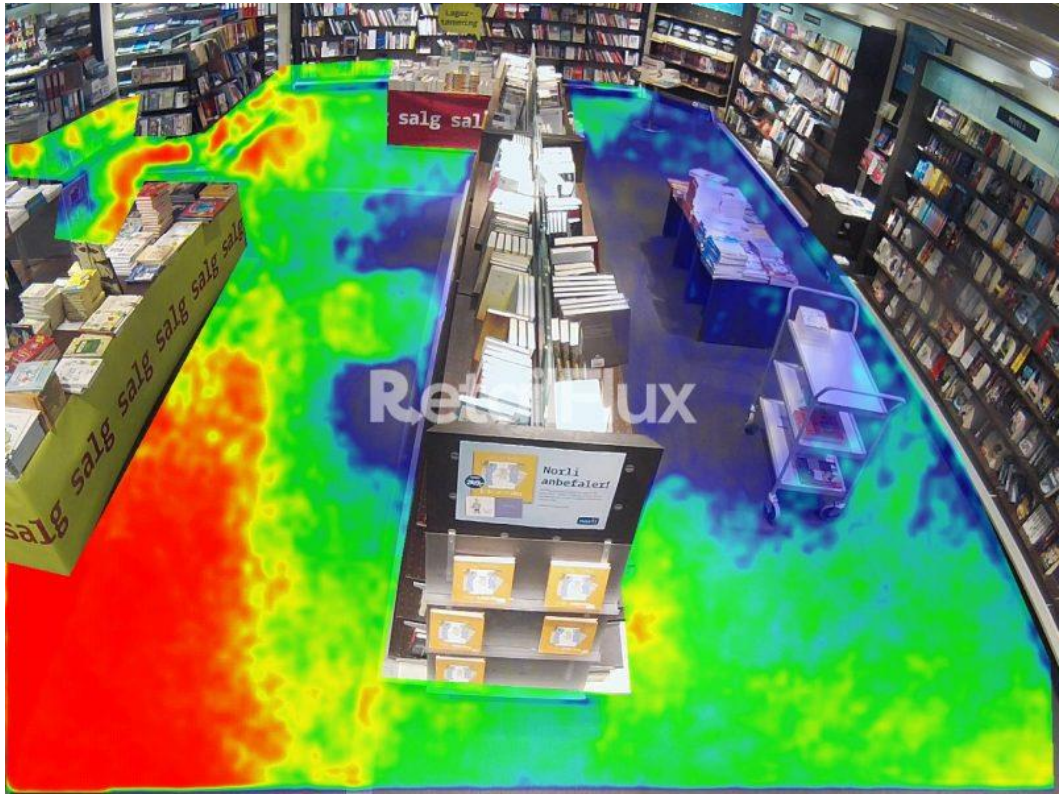
(a) (b) (c)

(d) (e) (f)

Figure 5 Tracking results for vehicles

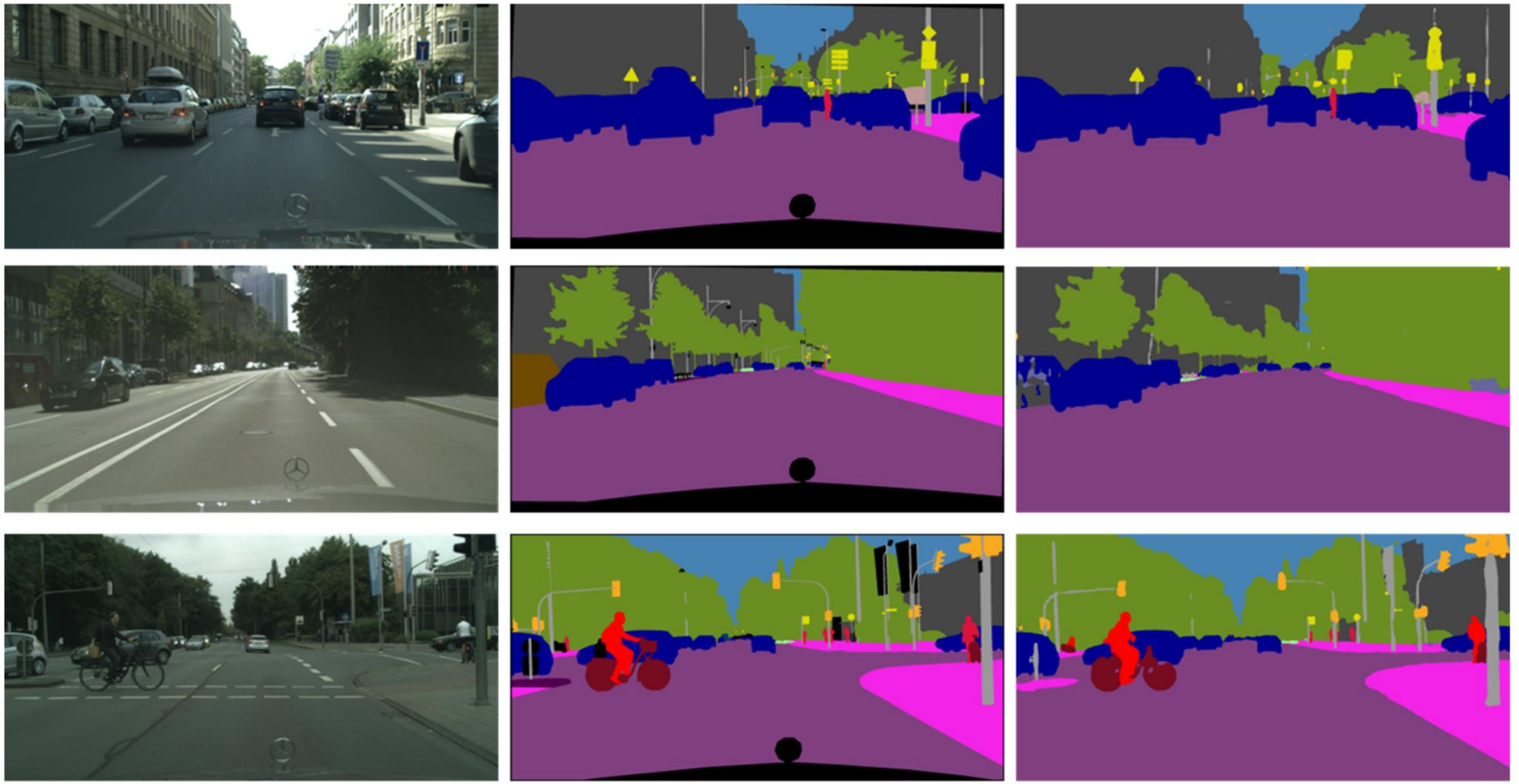
Store and Customer Analysis

Using object detection and tracking





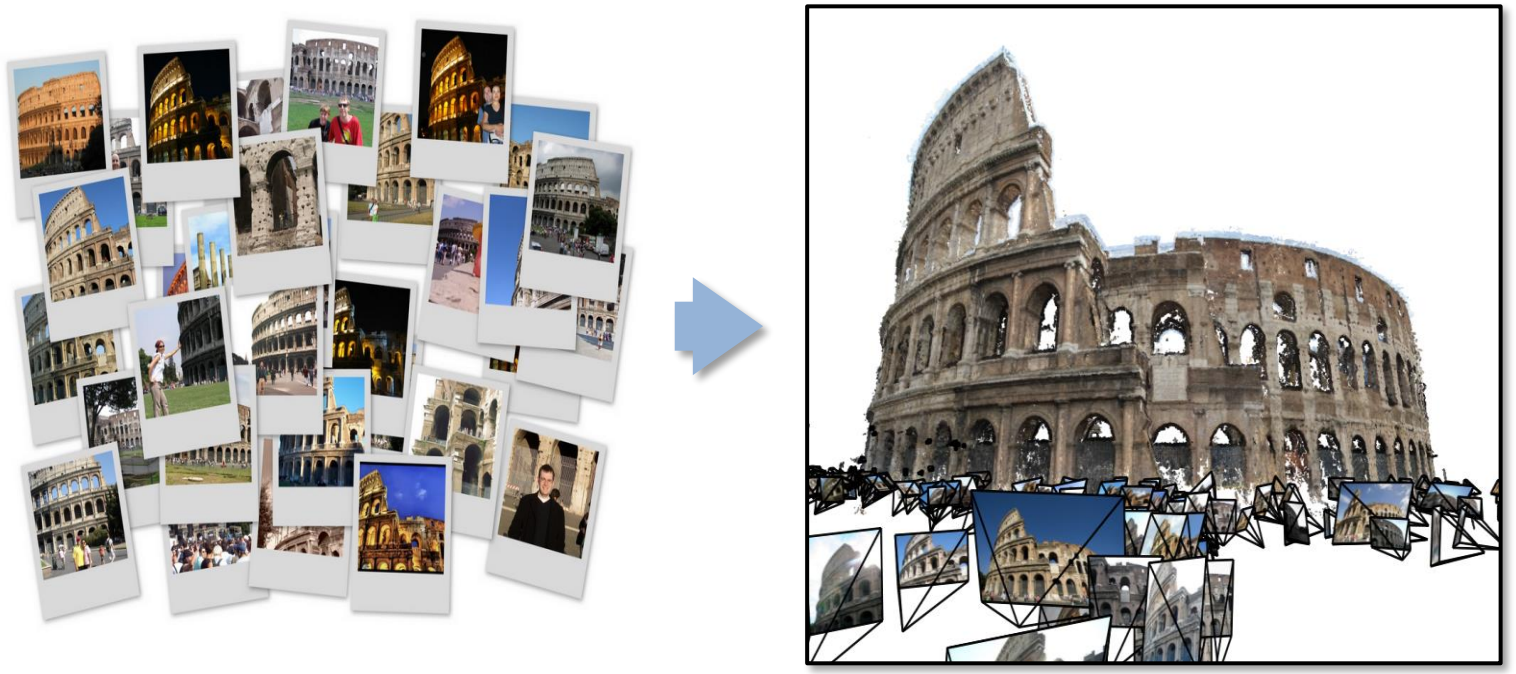
Scene Segmentation





Reconstructing the world

Reconstruct 3D scenes from huge collections of photos on Internet





Snavely, Seitz, Szeliski. **Reconstructing the World from Internet Photo Collections.** IJCV 2007

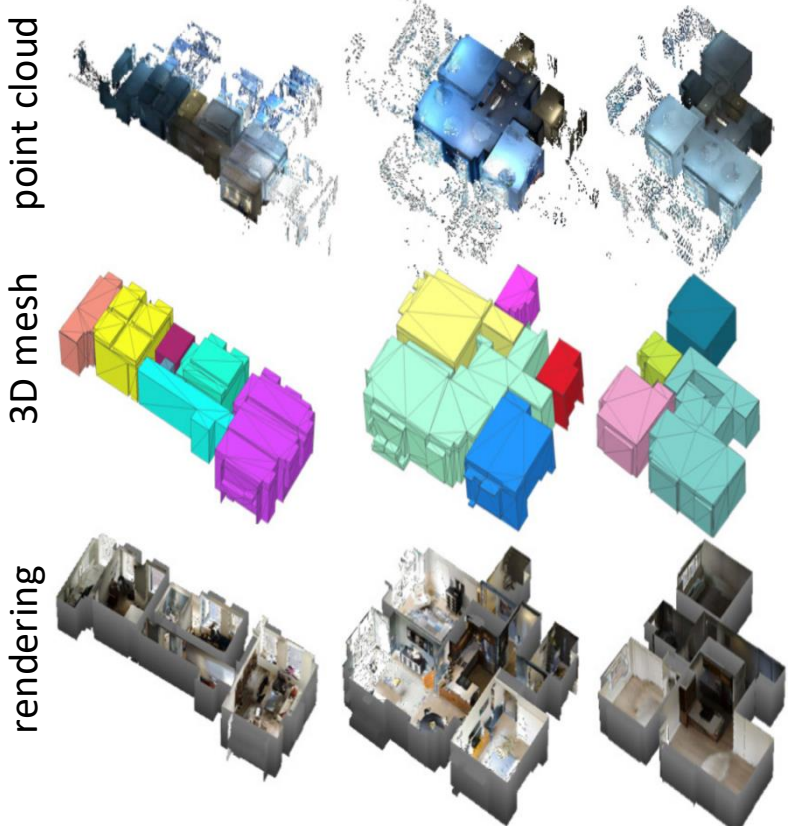
Reconstruct the great indoor

Depth Cameras



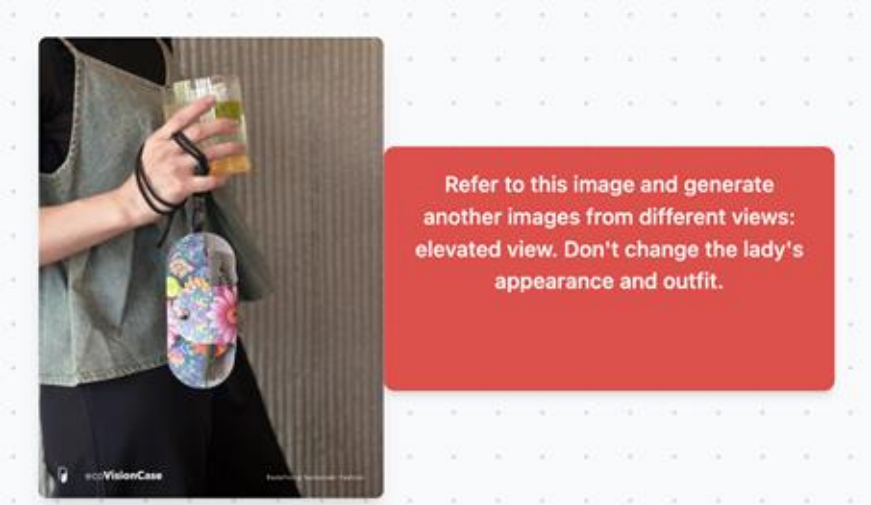
Robust Reconstruction of Indoor Scenes. CVPR 2015

Semantic Reconstruction



Structured Indoor Modeling. ICCV 2015

Nano Banana: changing perspective





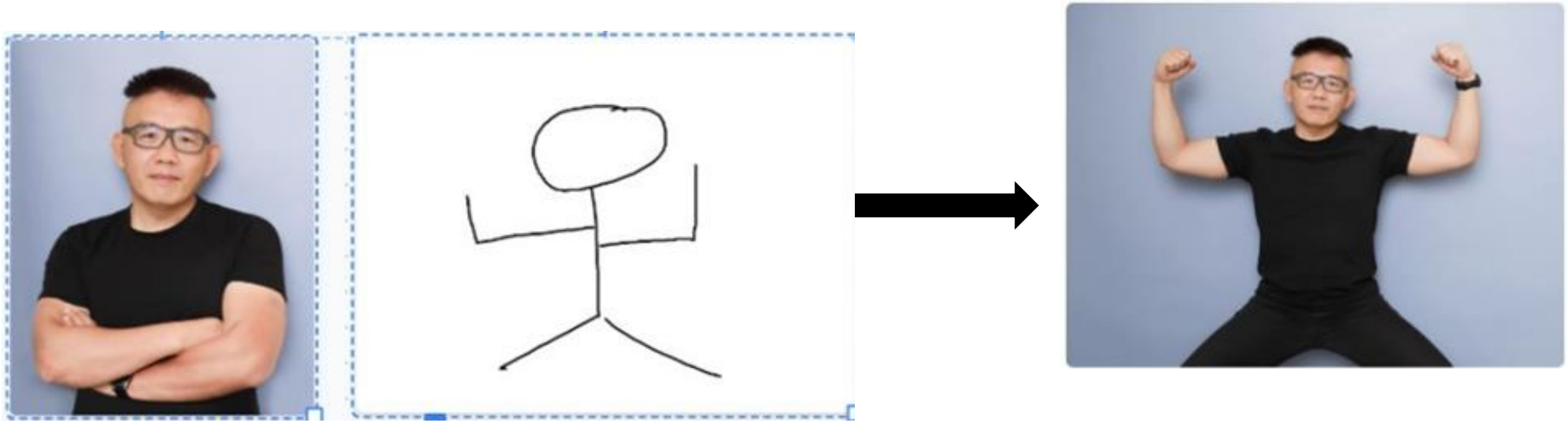
Nano Banana: Change Clothes

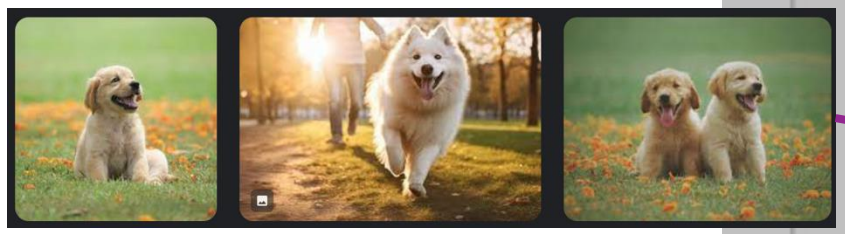






Nano Banana: Pose Control

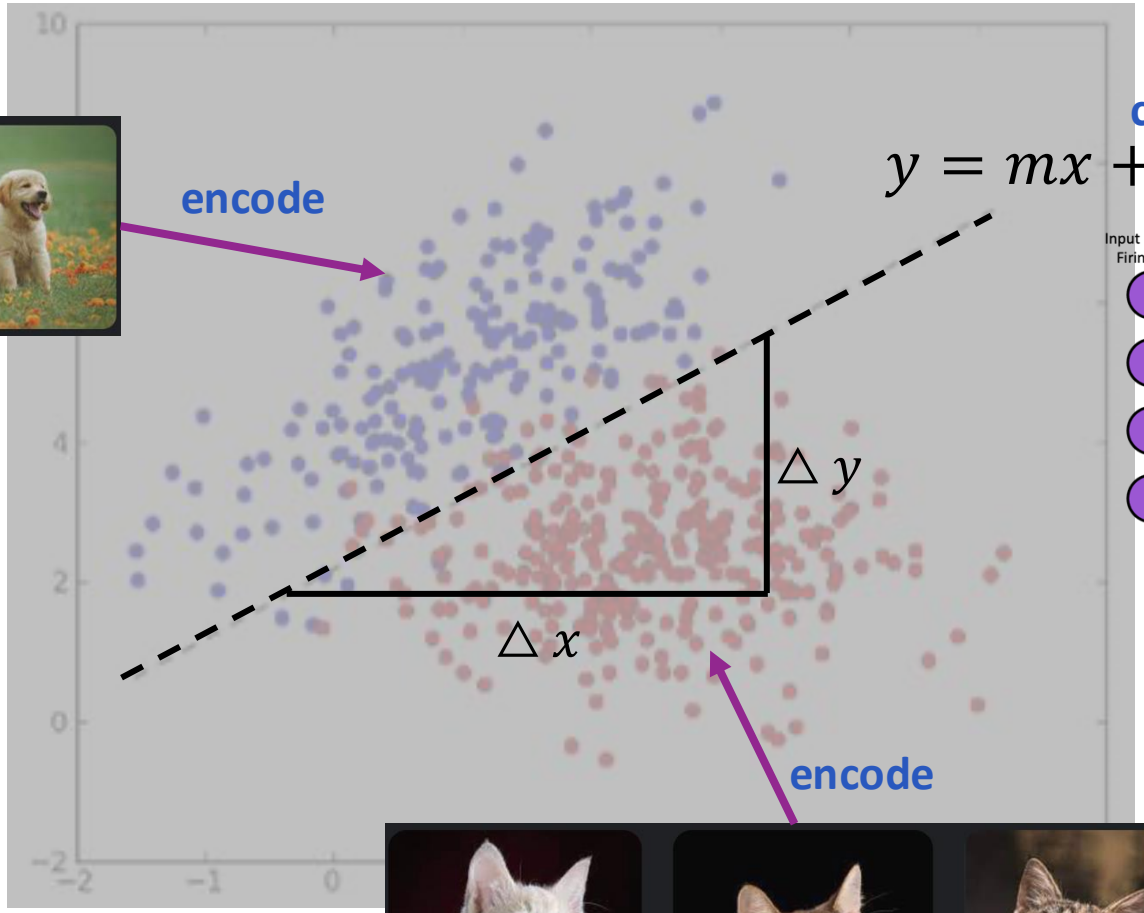


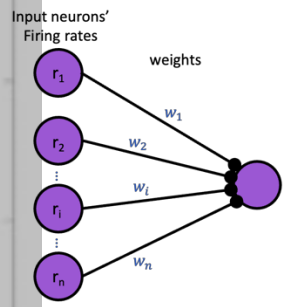


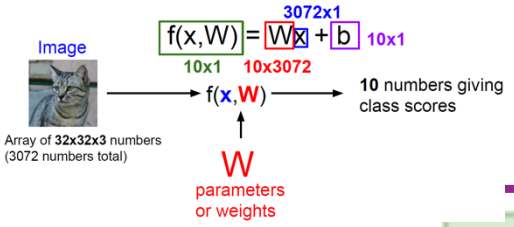
encode

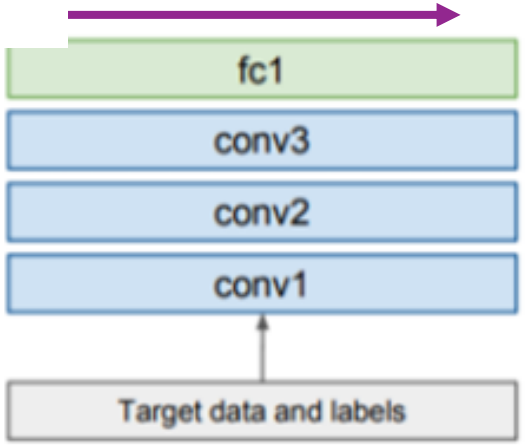
classifier

$y = mx + c$

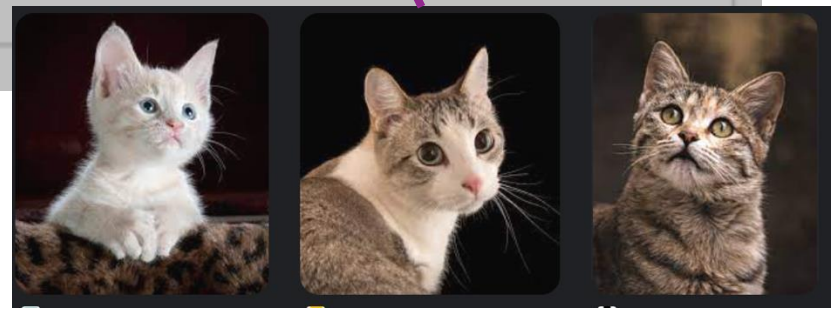








encode



Pre-train model (encoder)

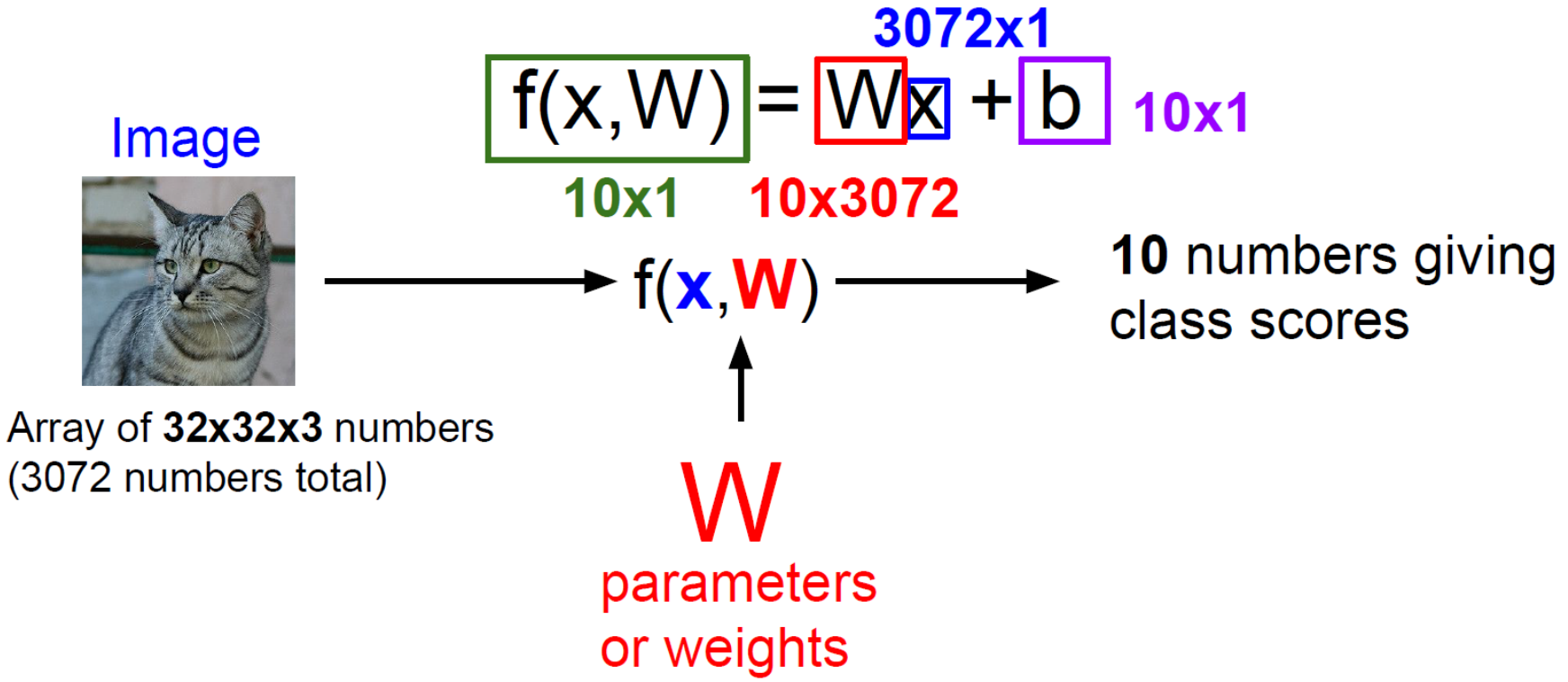
How to extract the features?

How to we compute all theses?



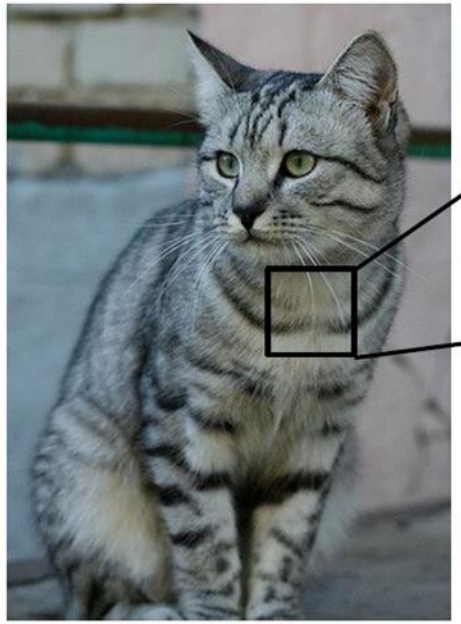
Recall: Image Classification with Linear Classifier

Training is to find **W** such that $f(x,\mathbf{W})$ gives the desired output, for an input image, x





The Problem: Semantic Gap



This image by Nikita is licensed under CC-BY 2.0

```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 120 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

What the computer sees

An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)



Examples: how computer classifies an image

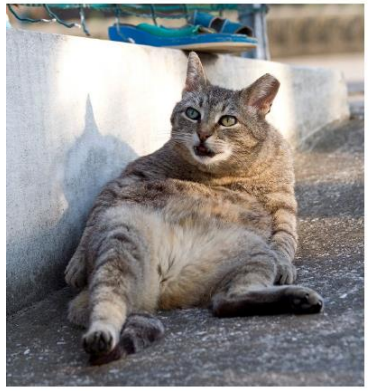
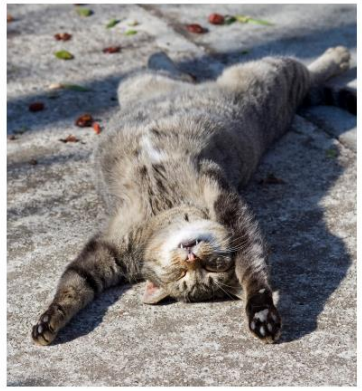
The Problem: Semantic Gap




Different backgrounds




Illumination

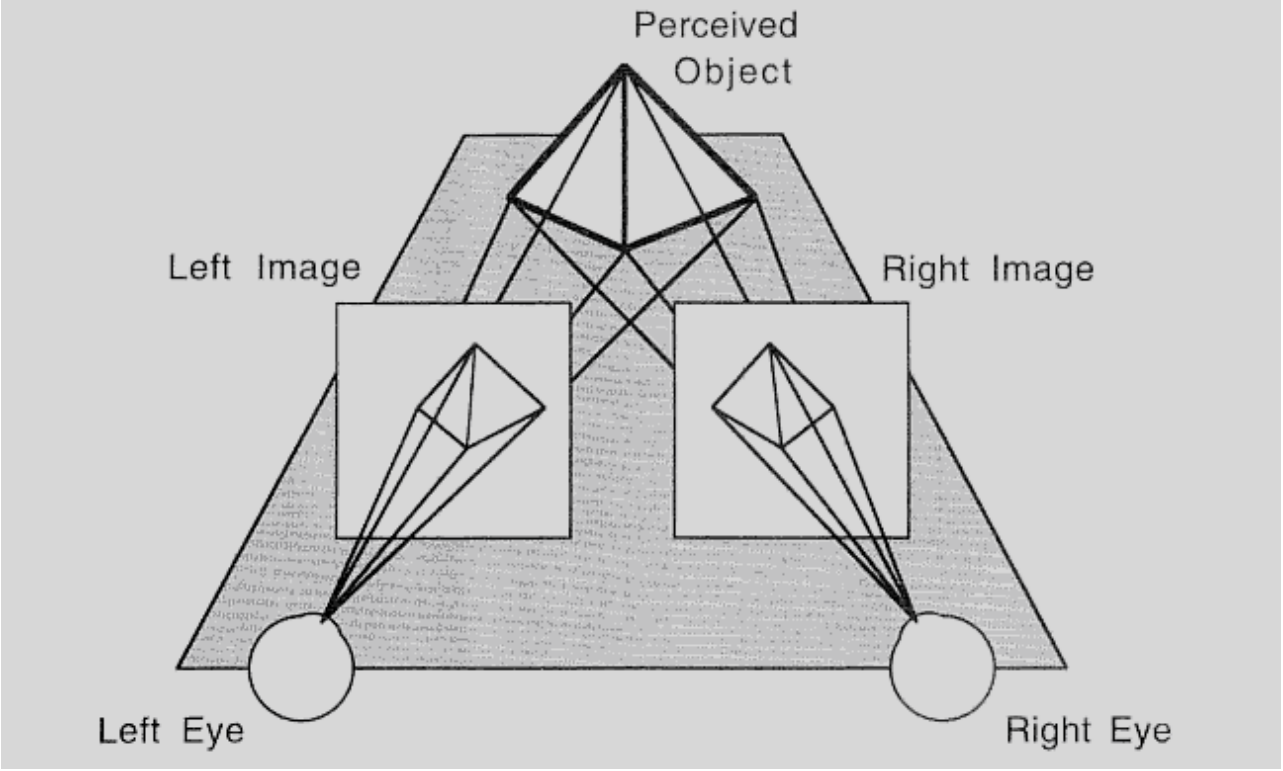
Deformation




Occlusion

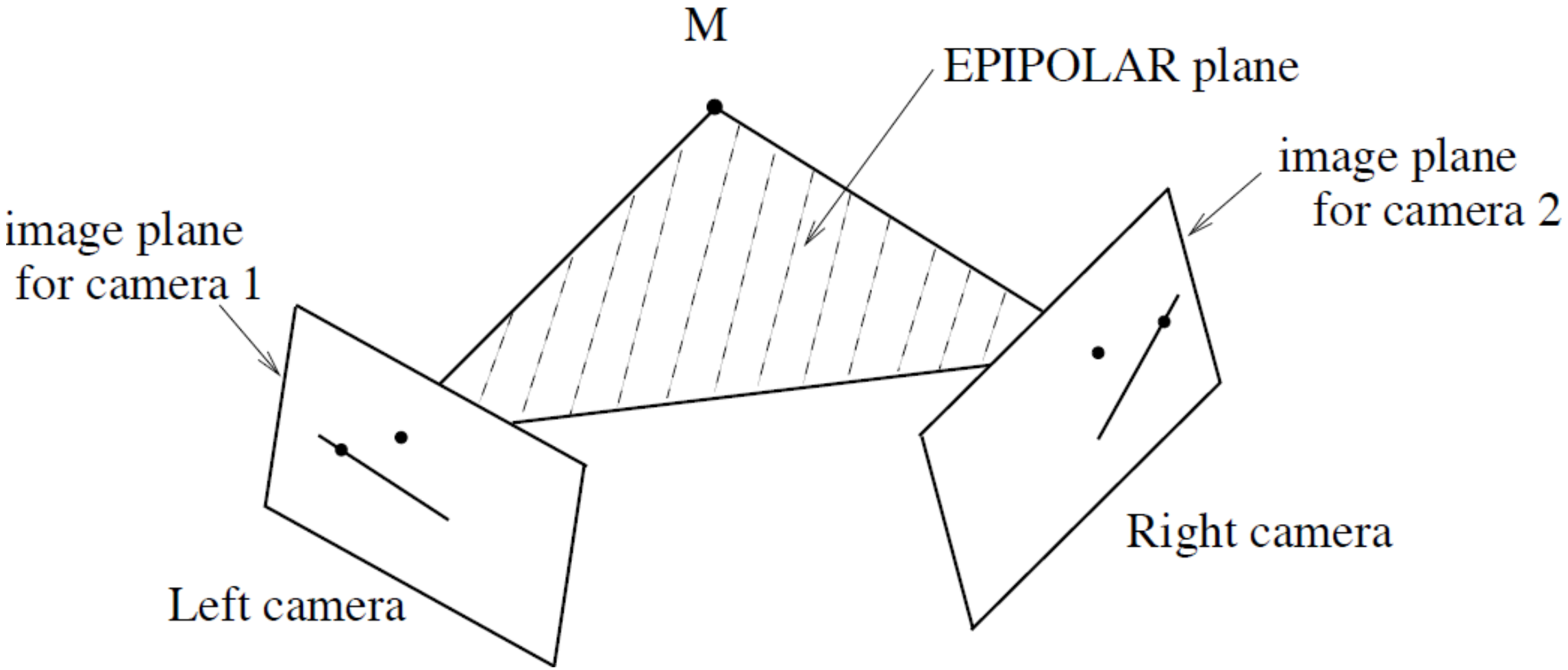
Old CV Model: Binocular Stereopsis

- Stereo images to reconstruct the world



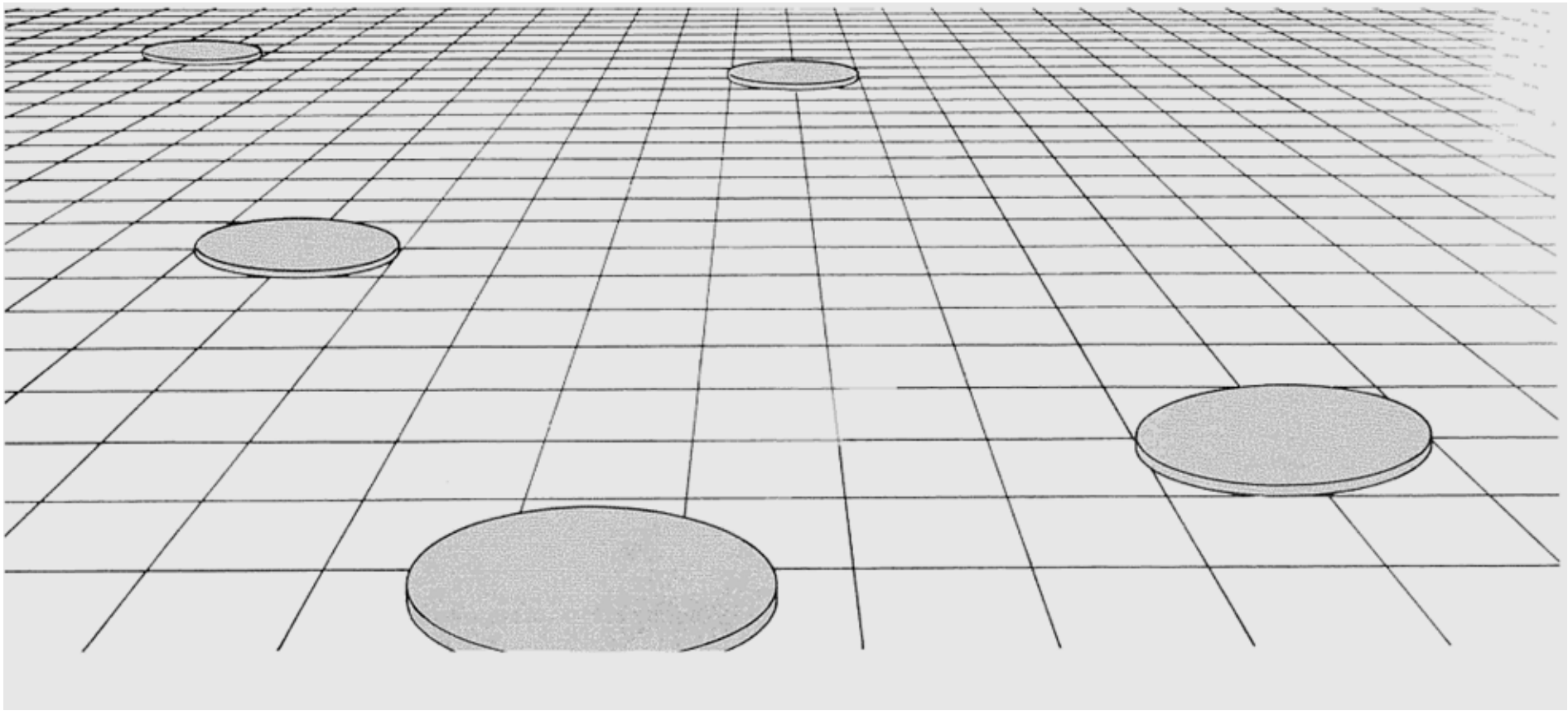


Old CV Model: Epipolar geometry



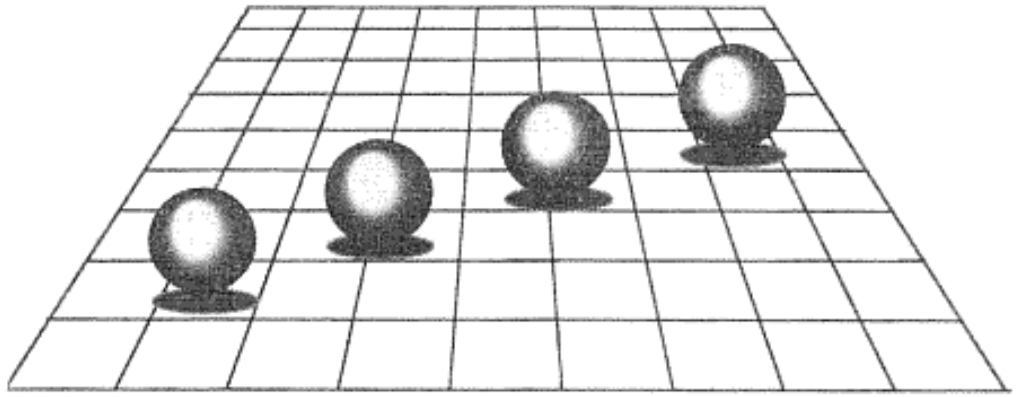


Old CV Model: Some Pictorial Cues

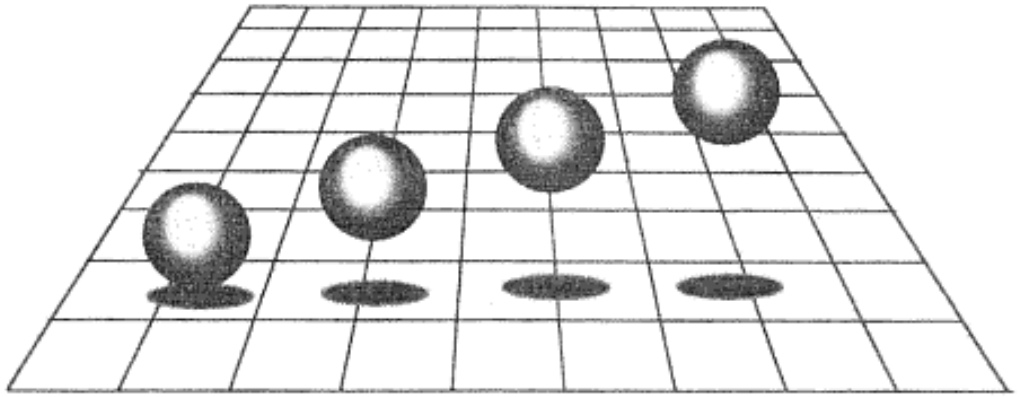




Old CV Model: Cast Shadows



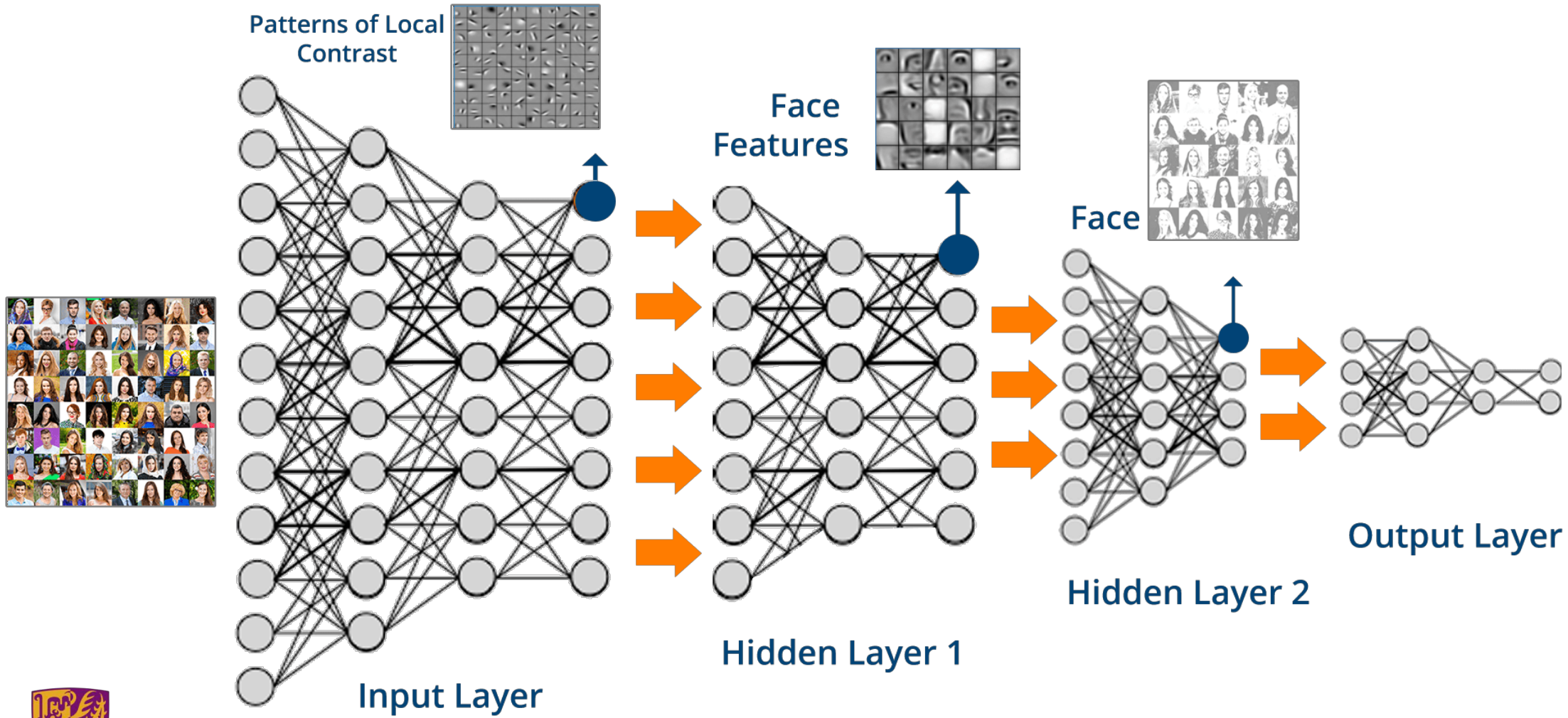
A



B



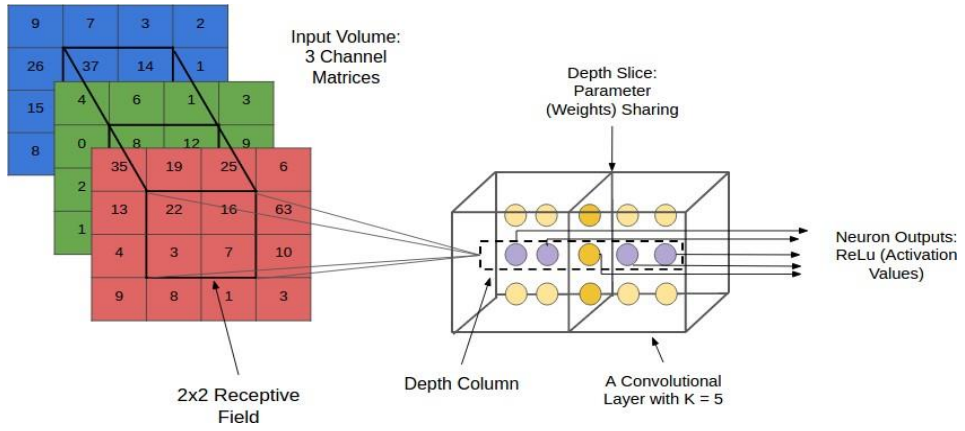
In Era of Deep Learning



Convolutional Layer

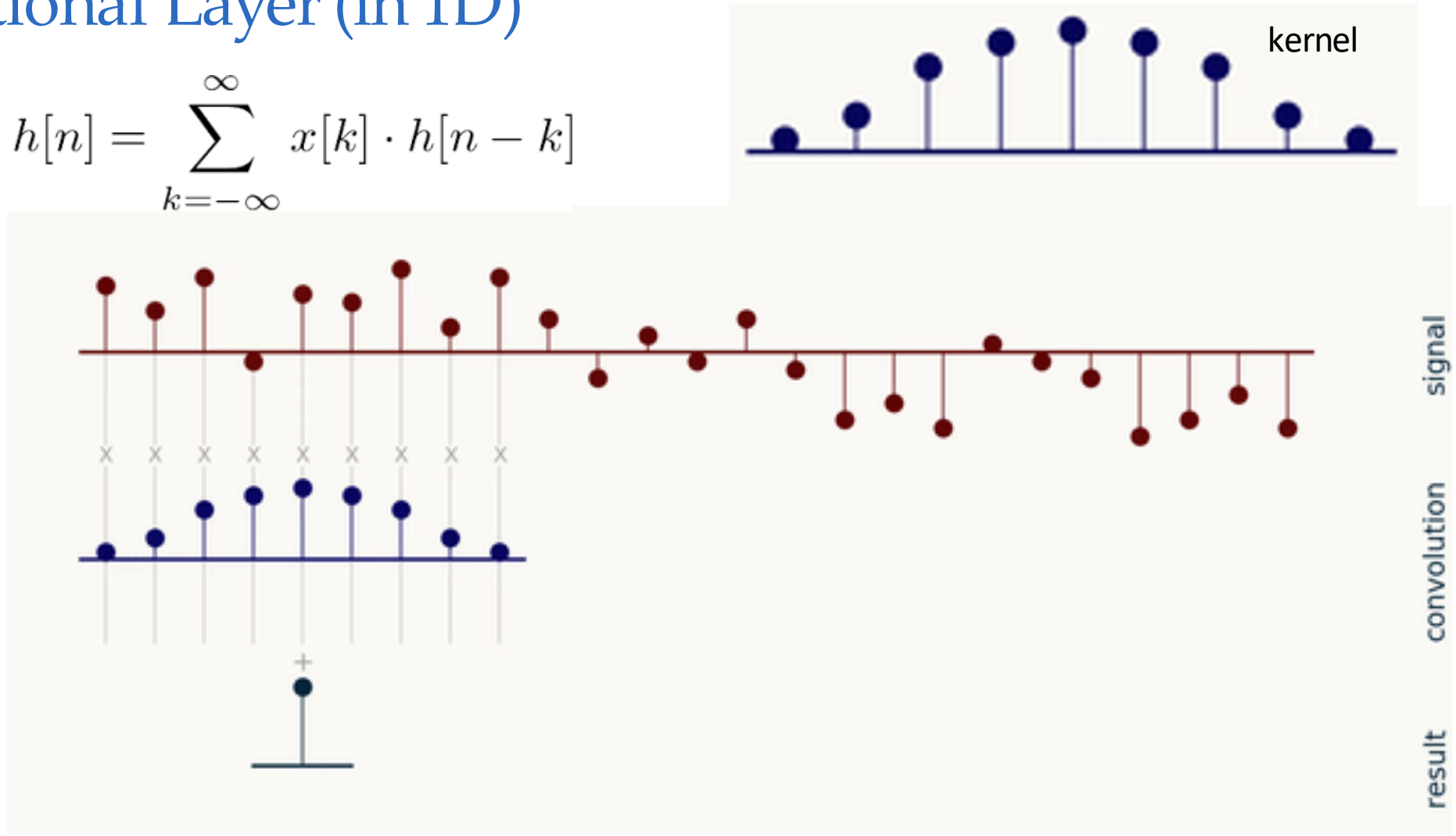
- Connected only to units in a small patch (**receptive field**) of the previous layer
- The receptive field is local in space
- Always full along the entire depth (e.g., RGB) of the input

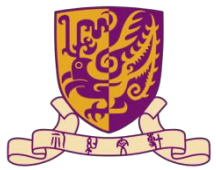
$$y[m,n] = x[m,n] * h[m,n] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[i,j] \cdot h[m-i, n-j]$$



Convolutional Layer (in 1D)

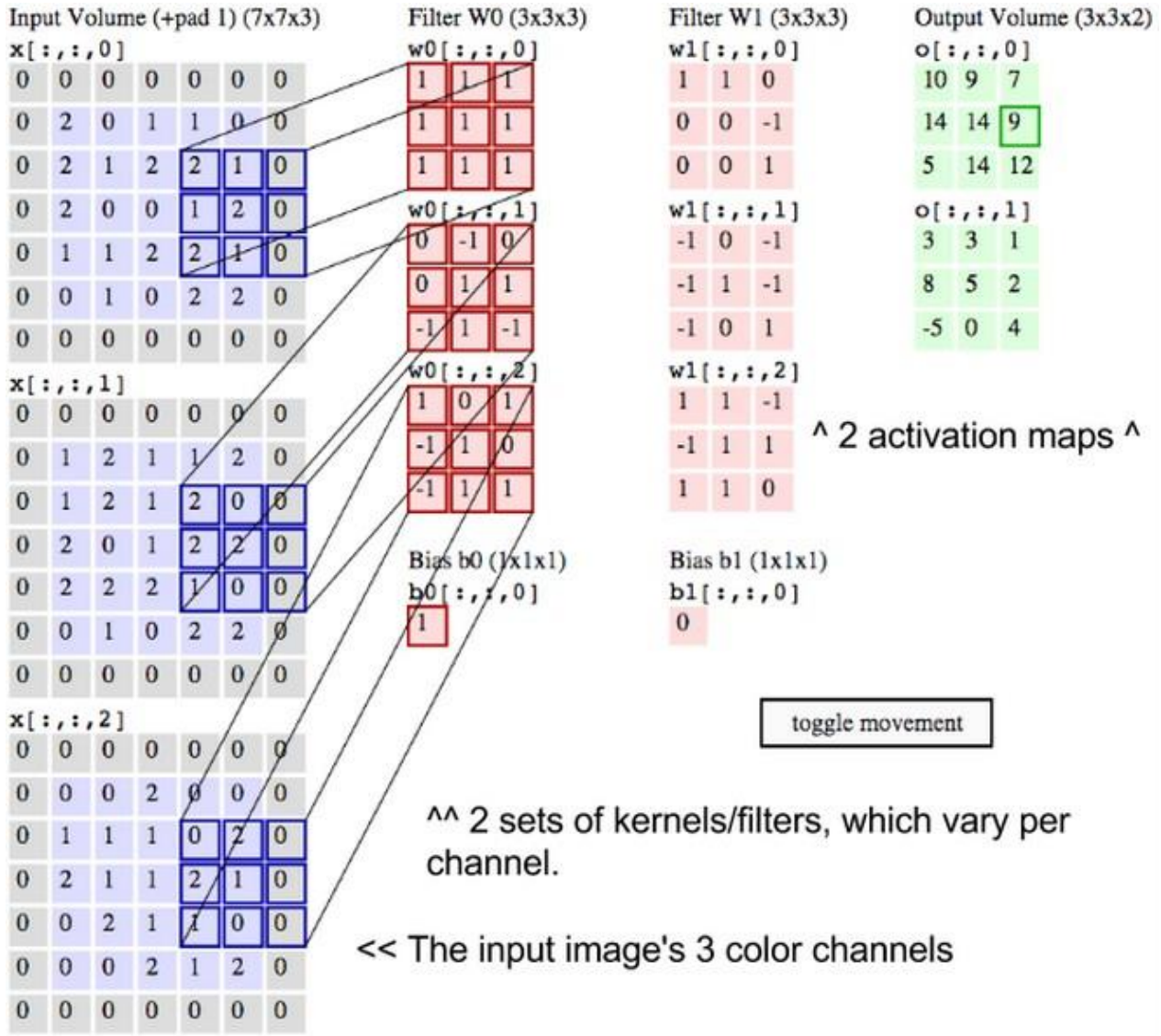
$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$





Convolutional Layer

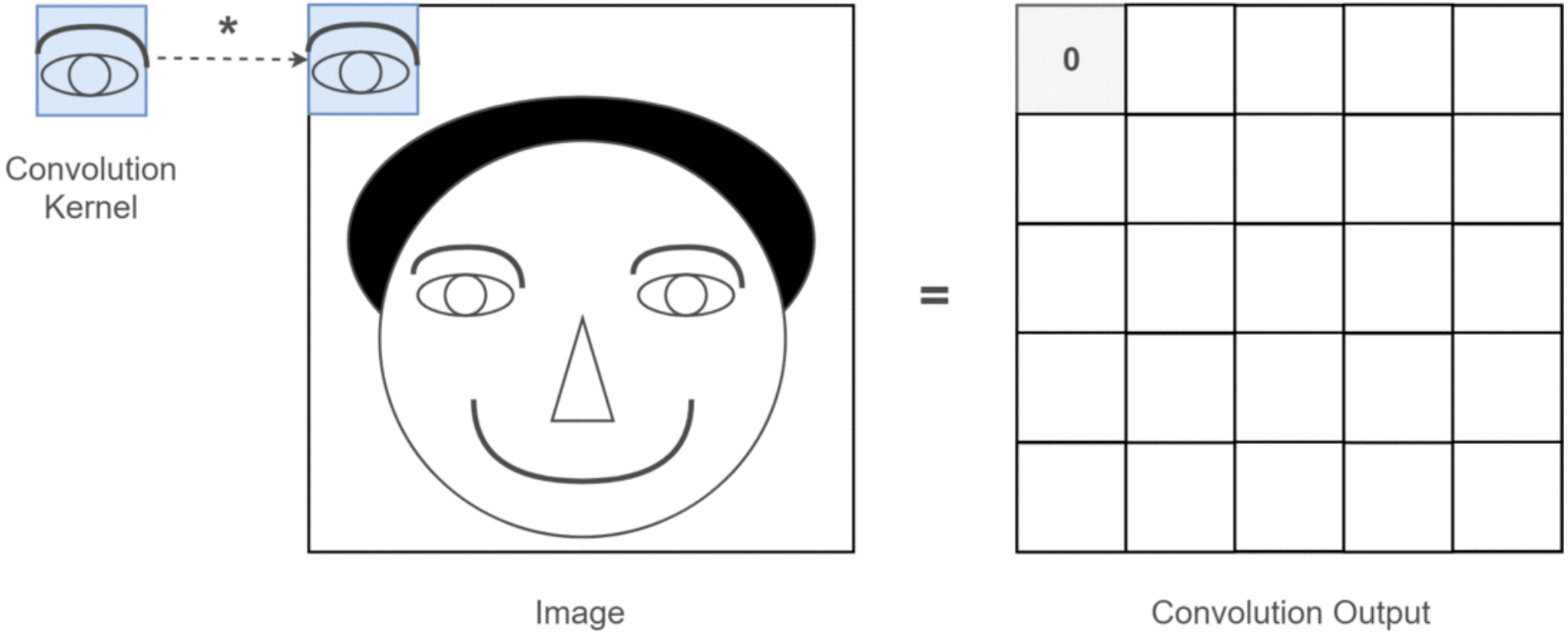
- move the filter 1 pixel at a time from left to right and top to bottom

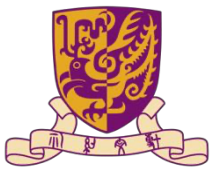




Convolutional Layer

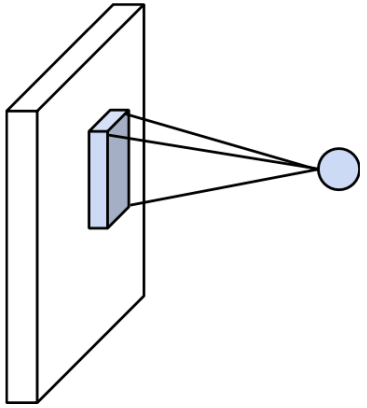
- More on this in the next lecture



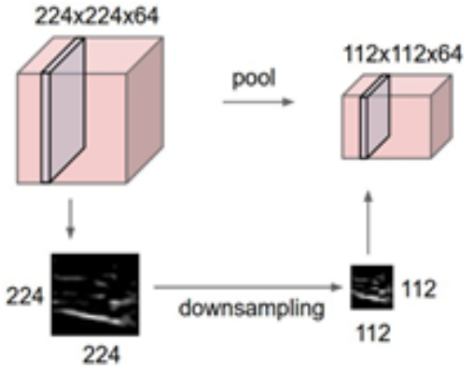


Components of CNNs

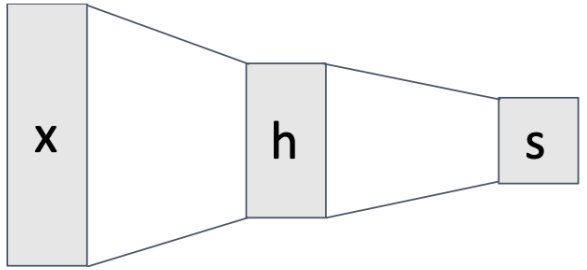
Convolution Layers



Pooling Layers



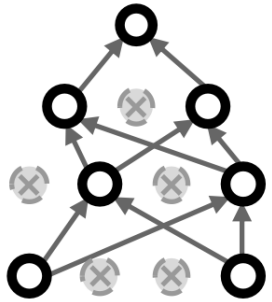
Fully-Connected Layers



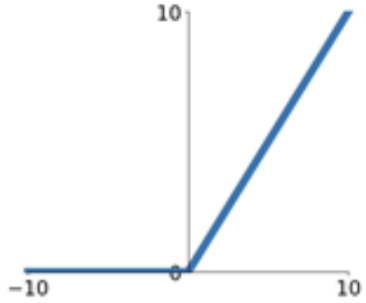
Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Dropout (sometimes)



Activation Functions



Agenda

- Computer Vision (CV) and Convolutional Neural Networks (CNN)
- Implementation of CV (OpenCV+PyTorch)
- Multi-model LLM and VLM



Implementation of CV

- OpenCV: provides a real-time optimized Computer Vision library, tools, and hardware

- 2500 optimized algorithms focusing real-time applications (ex: DIP)

- Cross-Platform C++, Python and Java on Linux, MacOS, Windows, iOS, and Android.



Examples of OpenCV

```python
import numpy as np
import cv2

img = cv2.imread('/path_to_image/img.jpg',-1)
scale_percent = 60 # percent of original size
width = int(img.shape[1] * scale_percent / 100)
height = int(img.shape[0] * scale_percent / 100)
dim = (width, height)
resized = cv2.resize(img, dim, interpolation = cv2.INTER_AREA)


# show image
cv2.imshow('image',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

```python
import cv2
import numpy as np

# Create a VideoCapture object and read from input file
# If the input is the camera, pass 0 instead of the video file name
cap = cv2.VideoCapture('/path_to_video/video.mp4')

# Check if camera opened successfully
if (cap.isOpened()== False):
  print("Error opening video stream or file")

# Read until video is completed
while(cap.isOpened()):
  # Capture frame-by-frame
  ret, frame = cap.read()
  if ret == True:

    # Display the resulting frame
    cv2.imshow('Frame',frame)

    # Press Q on keyboard to  exit
    if cv2.waitKey(25) & 0xFF == ord('q'):
      break

  # Break the loop
  else:
    break

# When everything done, release the video capture object
cap.release()

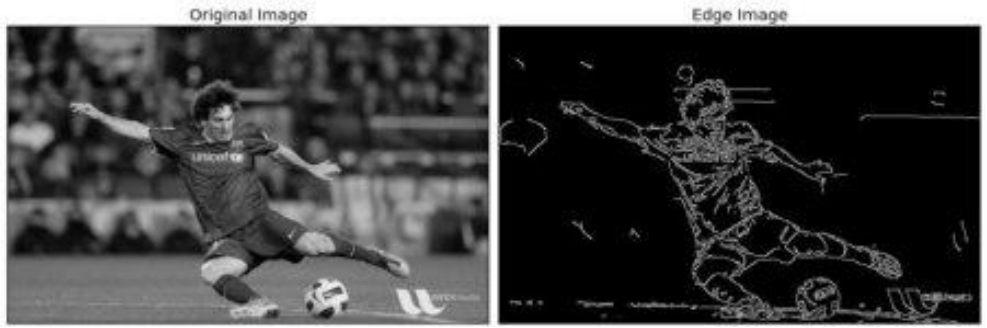
# Closes all the frames
cv2.destroyAllWindows()
```



Examples of OpenCV

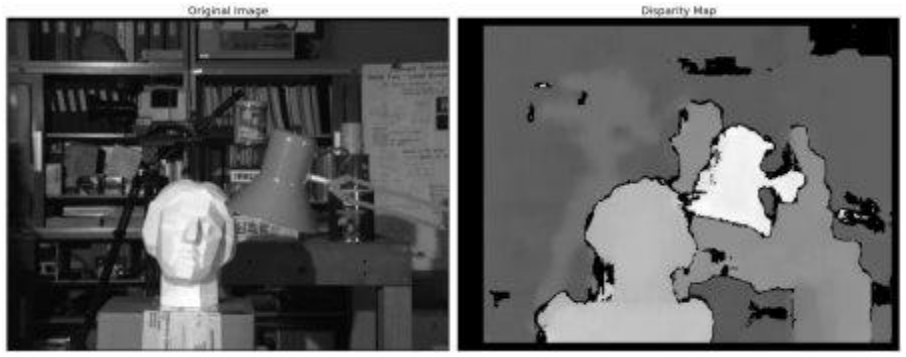
Edge detection

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('messi5.jpg',0)
edges = cv.Canny(img,100,200)
plt.subplot(121),plt.imshow(img,cmap = 'gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(edges,cmap = 'gray')
plt.title('Edge Image'), plt.xticks([]), plt.yticks([])
plt.show()
```



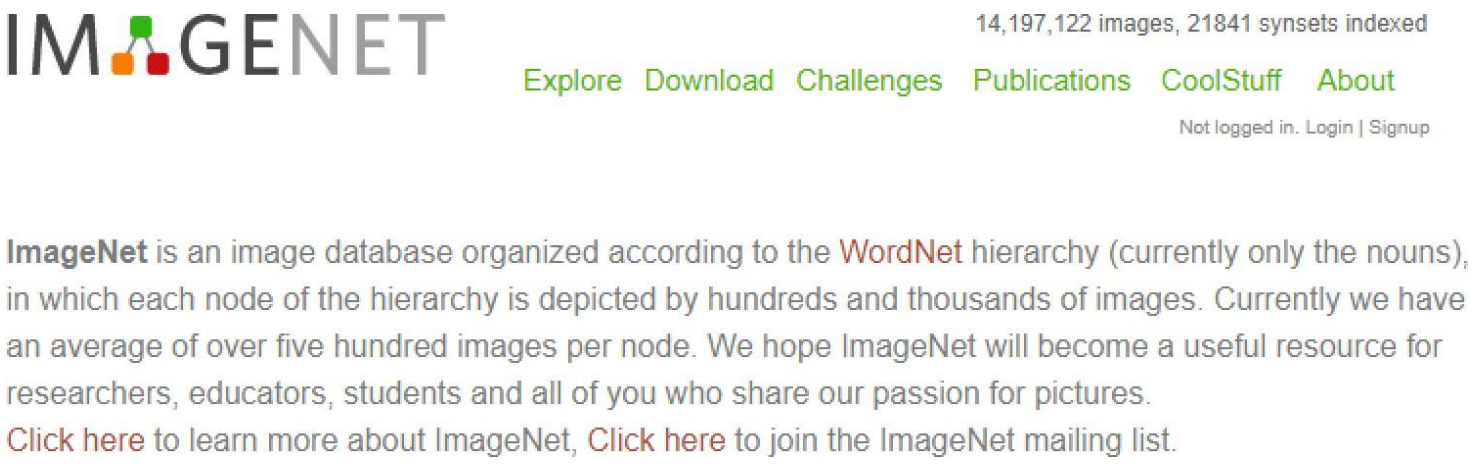
3D depth map

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
imgL = cv.imread('tsukuba_l.png',0)
imgR = cv.imread('tsukuba_r.png',0)
stereo = cv.StereoBM_create(numDisparities=16, blockSize=15)
disparity = stereo.compute(imgL,imgR)
plt.imshow(disparity,'gray')
plt.show()
```



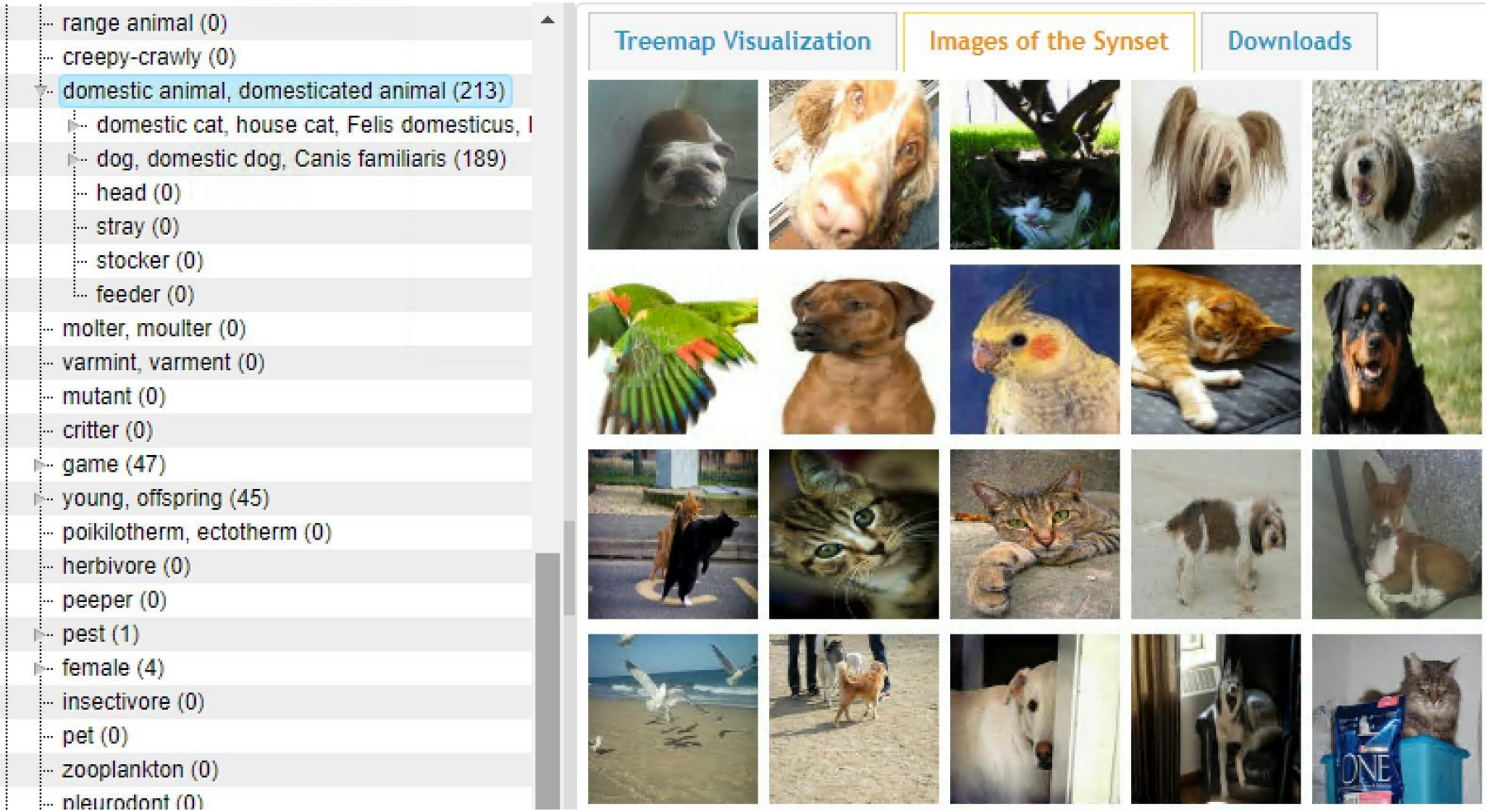
ImageNet [1]

- The ImageNet project is a large visual database designed for use in visual object recognition software research.
- 14 million images have been hand-annotated by ImageNet to indicate what objects are pictured



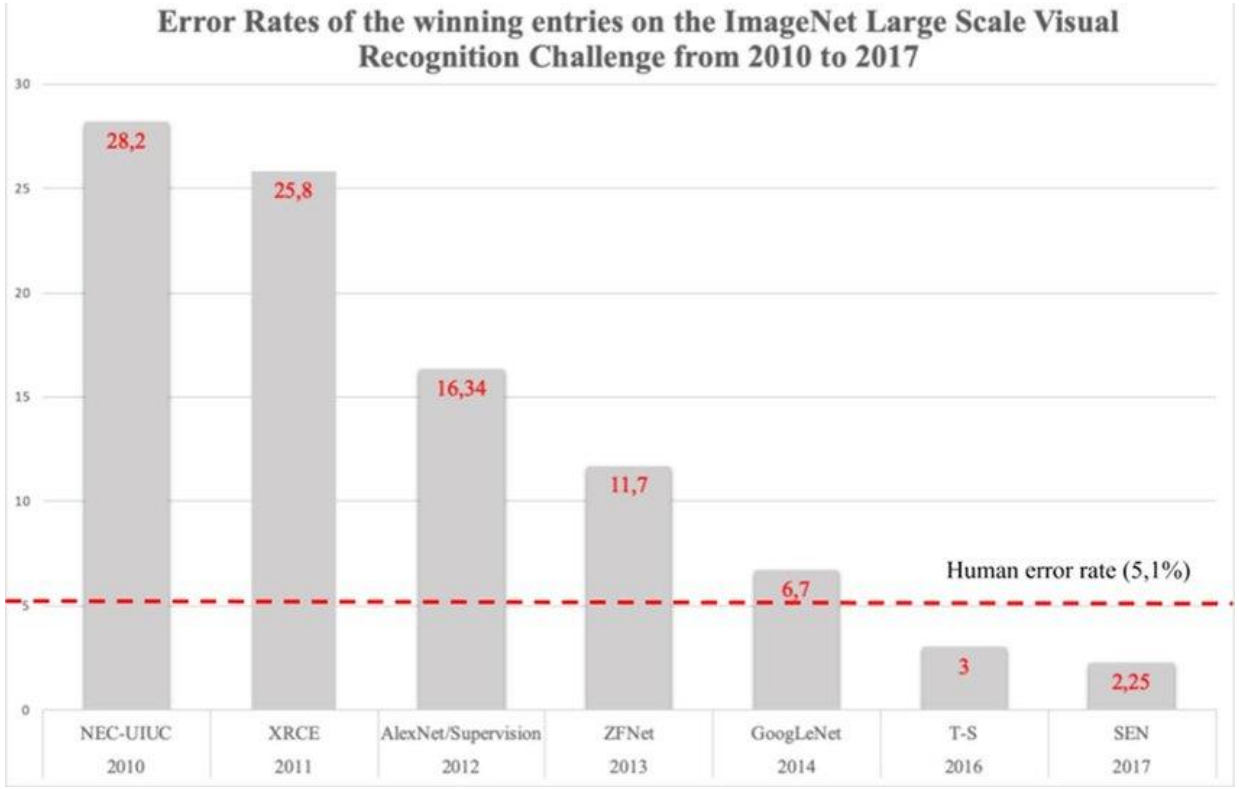
[1] https://www.image-net.org

ImageNet


range animal (0)
creepy-crawly (0)
domestic animal, domesticated animal (213)
domestic cat, house cat, Felis domesticus, I
dog, domestic dog, Canis familiaris (189)
head (0)
stray (0)
stocker (0)
feeder (0)
molter, moulter (0)
varmint, varment (0)
mutant (0)
critter (0)
game (47)
young, offspring (45)
poikilotherm, ectotherm (0)
herbivore (0)
peeper (0)
pest (1)
female (4)
insectivore (0)
pet (0)
zooplankton (0)
pleurodont (0)
Treemap Visualization
Images of the Synset
Downloads

Large Scale Visual Recognition Challenge

- Since 2010, the ImageNet project runs an annual software contest, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC)



Popular CNN Architectures

- Pre-trained version of the models can be downloaded from:
  https://keras.io/api/applications/

Structure	# Parameters (M)	Storage Space (MB)
VGG16	138	500
ResNet50	25	98
ResNet101	44	171
InceptionV3	24	92
Xception	23	88
DenseNet121	8	33
DenseNet169	14	57
DenseNet201	20	80
EfficientNetB0	5.3	20
EfficientNetB2	9	36

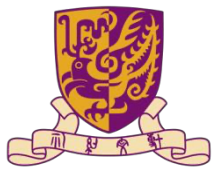


Image Classification with (ResNet)

```
class ResNetFeatureExtractor(nn.Module):
    def __init__(self):
        super(ResNetFeatureExtractor, self).__init__()
        self.model = models.resnet50(pretrained=True)
        # Remove the final layer
        self.model = nn.Sequential(*list(self.model.childr

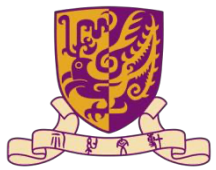
    def forward(self, x):
        x = self.model(x)
        return x.view(x.size(0), -1)  # Flatten the output
```

```
with torch.no_grad():
    for file in files_cat:
        image_path2 = data_path + 'cat/' + file
        output = model(load_image(image_path2))
        # convert to numpy
        output = output.cpu().numpy()
        array_emb_cat.append(output)

    array_emb_dog = []
    # encode all images in files
    for file in files_dog:
        image_path2 = data_path + 'dog/' + file
        output = model(load_image(image_path2))
        # convert to numpy
        output = output.cpu().numpy()
        array_emb_dog.append(output)

# build a KNN model to classify cat and dog
array_emb = array_emb_cat + array_emb_dog
labels = [0]*len(array_emb_cat) + [1]*len(array_emb_dog)
from sklearn.neighbors import KNeighborsClassifier

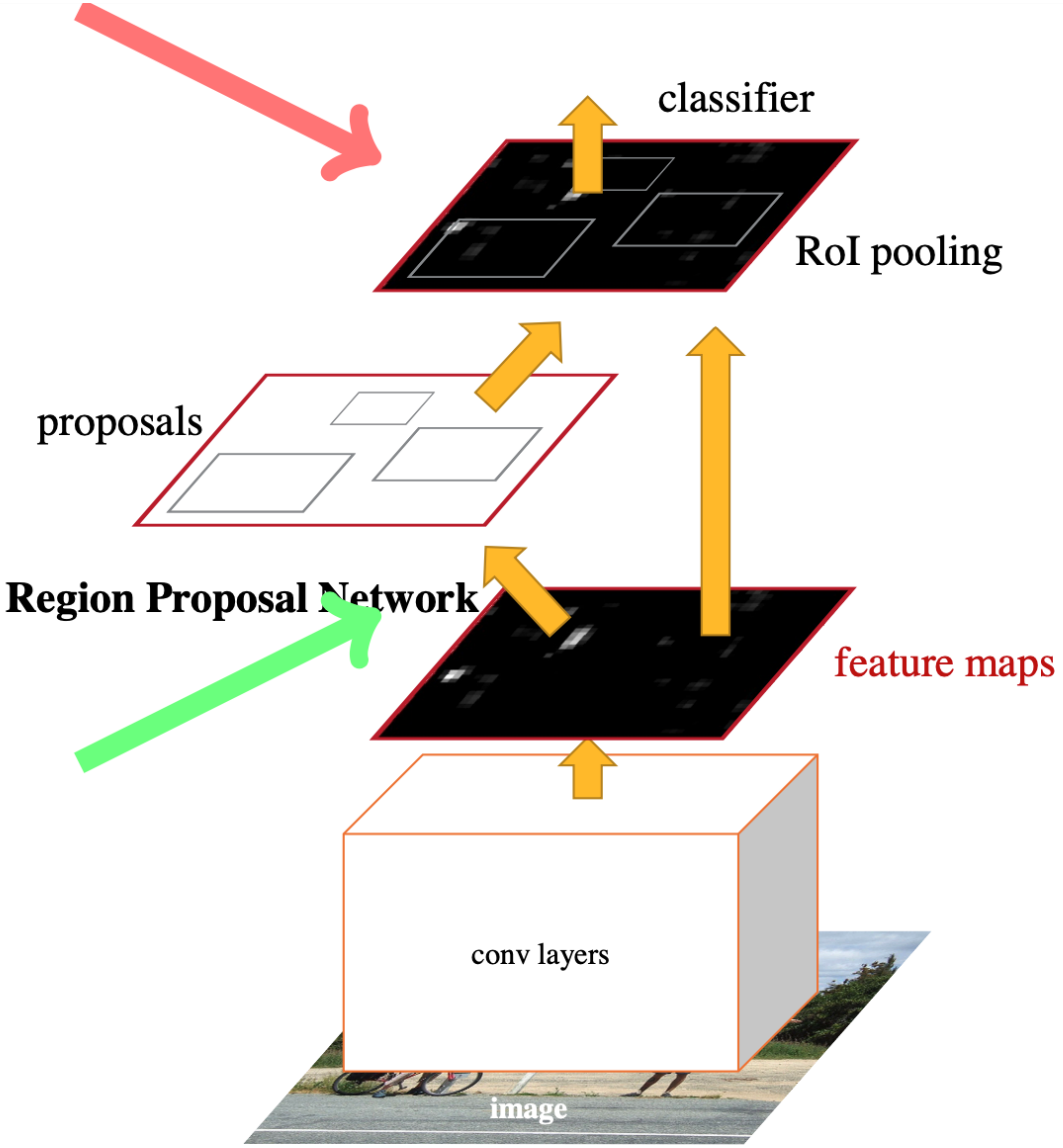
array_emb =np.vstack(array_emb)
neigh = KNeighborsClassifier(n_neighbors=10)
neigh.fit(array_emb, labels)
```



Yolo – You Only Look Once

One net to outputs:

1. Position
2. Height
3. Width
4. class and the probability



Add Yolo26 without NMS



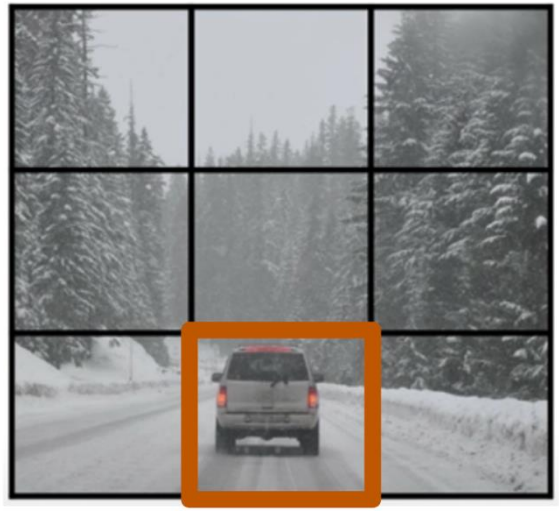
Ref: You Only Look Once: Unified, Real-Time Object Detection

Yolo Training

- YOLO is a regression algorithm. What is X? What is Y?
- X is simple, just an image width (in pixels) * height (in pixels) * RGB values
- Y is a tensor of size S*S*(B*5+C)
- B*5 + C term represents the predictions + class predicted distribution for a grid block

For each grid block, we have a vector like this. For this example B is 2 and C is 2

ρ_1
b_x_1
b_y_1
b_h_1
b_w_1
ρ_2
b_x_2
b_y_2
b_h_2
b_w_2
c_1
c_2



GT label example:

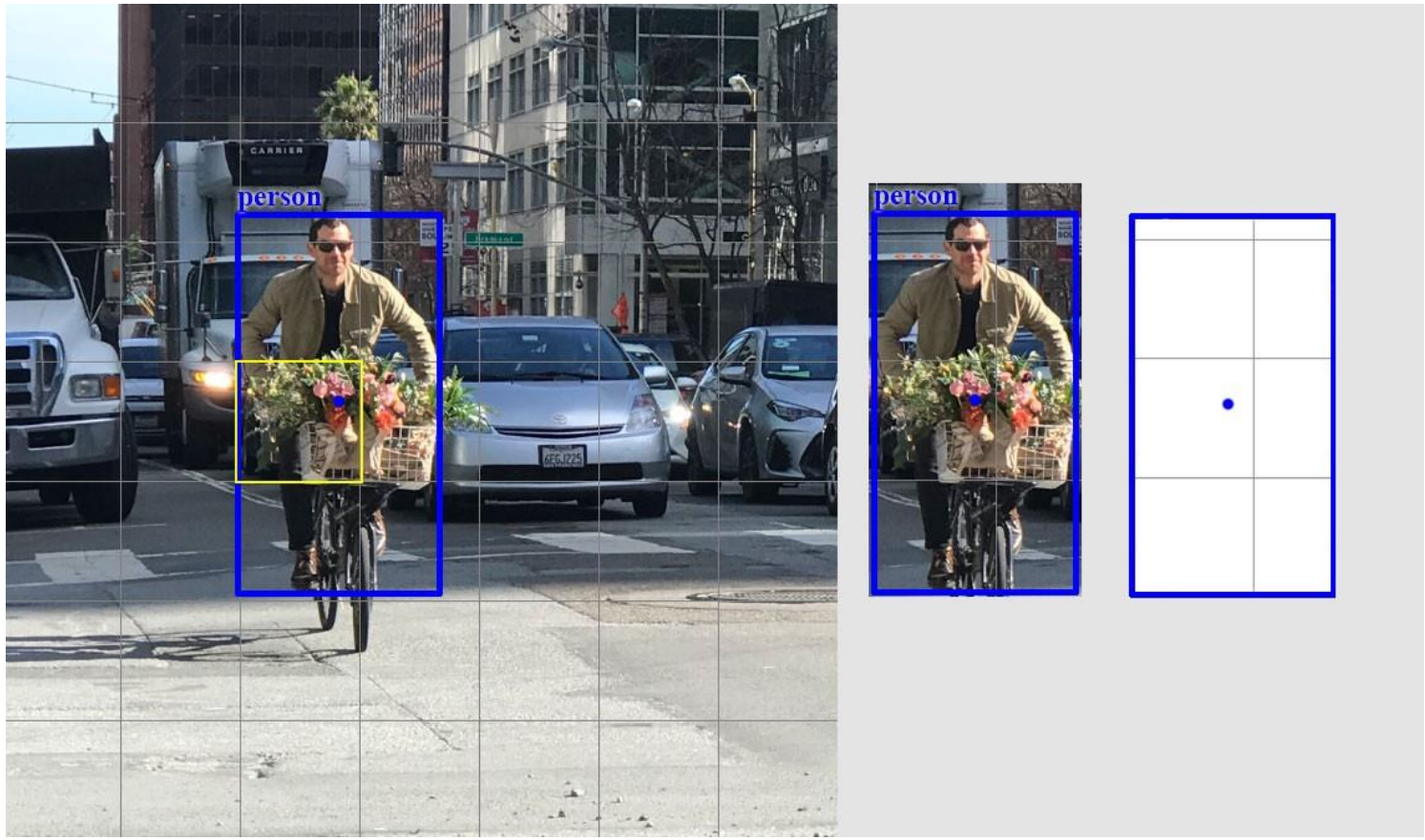
1
b_x_1
b_y_1
b_h_1
b_w_1
0
?
?
?
?
c_1 = 1
c_2 = 0

https://grok.com/share/c2hhcmQtMg%3D%3D_22452501-b501-4512-99fd-9f307134af50

Yolo

YOLO divides the input image into an **S**×**S** grid. Each grid cell predicts only **one** object.

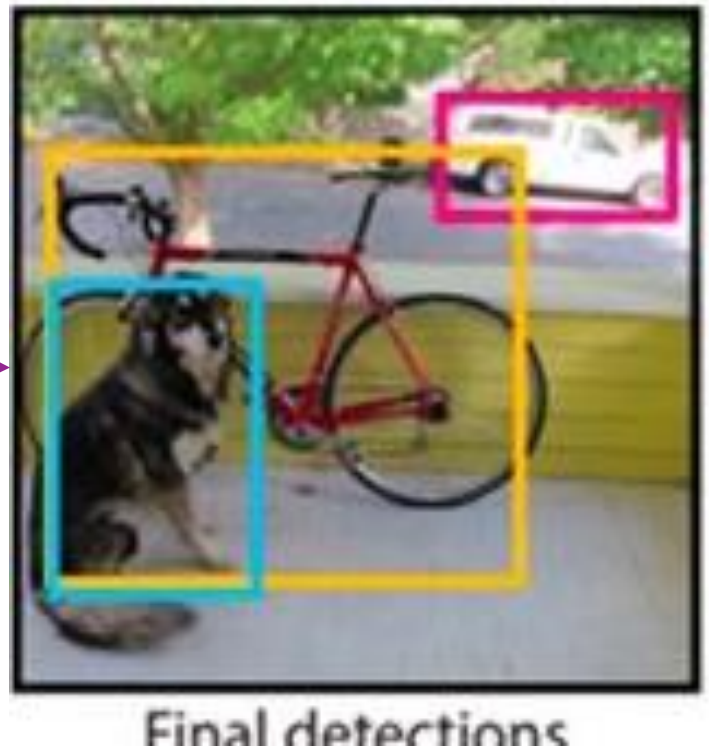
For example, the yellow grid cell below tries to predict the "person" object whose centre (the blue dot) falls inside the grid cell.



Yolo



Non-mean suppression

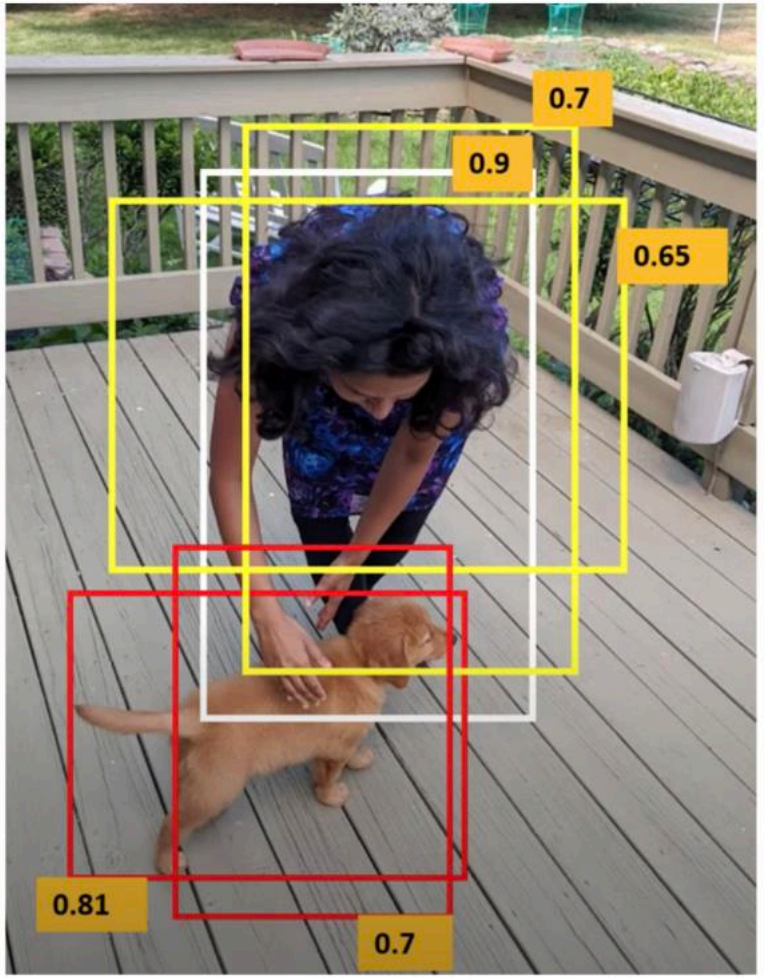


Non-mean suppression

- Most of the time objects fall in one grid, however it is still possible to get redundant boxes (rare case as object must be close to multiple grid cells for this to happen)

- Discard bounding box with high overlap (keeping the bounding box with highest confidence)

- ❖ Adds 2-3% on final mAP score



Yolo

```python
import cv2
import numpy as np

# function to get the output layer names
# in the architecture
def get_output_layers(net):
    layer_names = net.getLayerNames()
    output_layers = [layer_names[i[0] - 1] for i in net.getUnconnectedOutLayers()]
    return output_layers

# read input image
image = cv2.imread(args.image)
Width = image.shape[1]
Height = image.shape[0]
scale = 0.00392
# read class names from text file
classes = None
with open(args.classes, 'r') as f:
    classes = [line.strip() for line in f.readlines()]

# read pre-trained model and config file
net = cv2.dnn.readNet(args.weights, args.config)
# create input blob
blob = cv2.dnn.blobFromImage(image, scale, (416,416), (0,0,0), True, crop=False)
# set input blob for the network
net.setInput(blob)
# run inference through the network
# and gather predictions from output layers
outs = net.forward(get_output_layers(net))
```

```python
# initialization
class_ids = []
confidences = []
boxes = []
conf_threshold = 0.5
nms_threshold = 0.4

# for each detetion from each output layer
# get the confidence, class id, bounding box params
# and ignore weak detections (confidence < 0.5)
for out in outs:
    for detection in out:
        scores = detection[5:]
        class_id = np.argmax(scores)
        confidence = scores[class_id]
        if confidence > 0.5:
            center_x = int(detection[0] * Width)
            center_y = int(detection[1] * Height)
            w = int(detection[2] * Width)
            h = int(detection[3] * Height)
            x = center_x - w / 2
            y = center_y - h / 2
            class_ids.append(class_id)
            confidences.append(float(confidence))
            boxes.append([x, y, w, h])

# apply non-max suppression
indices = cv2.dnn.NMSBoxes(boxes, confidences, conf_threshold, nms_threshold)
```



Demo: Human Detection

- Detect human from an image using Yolo

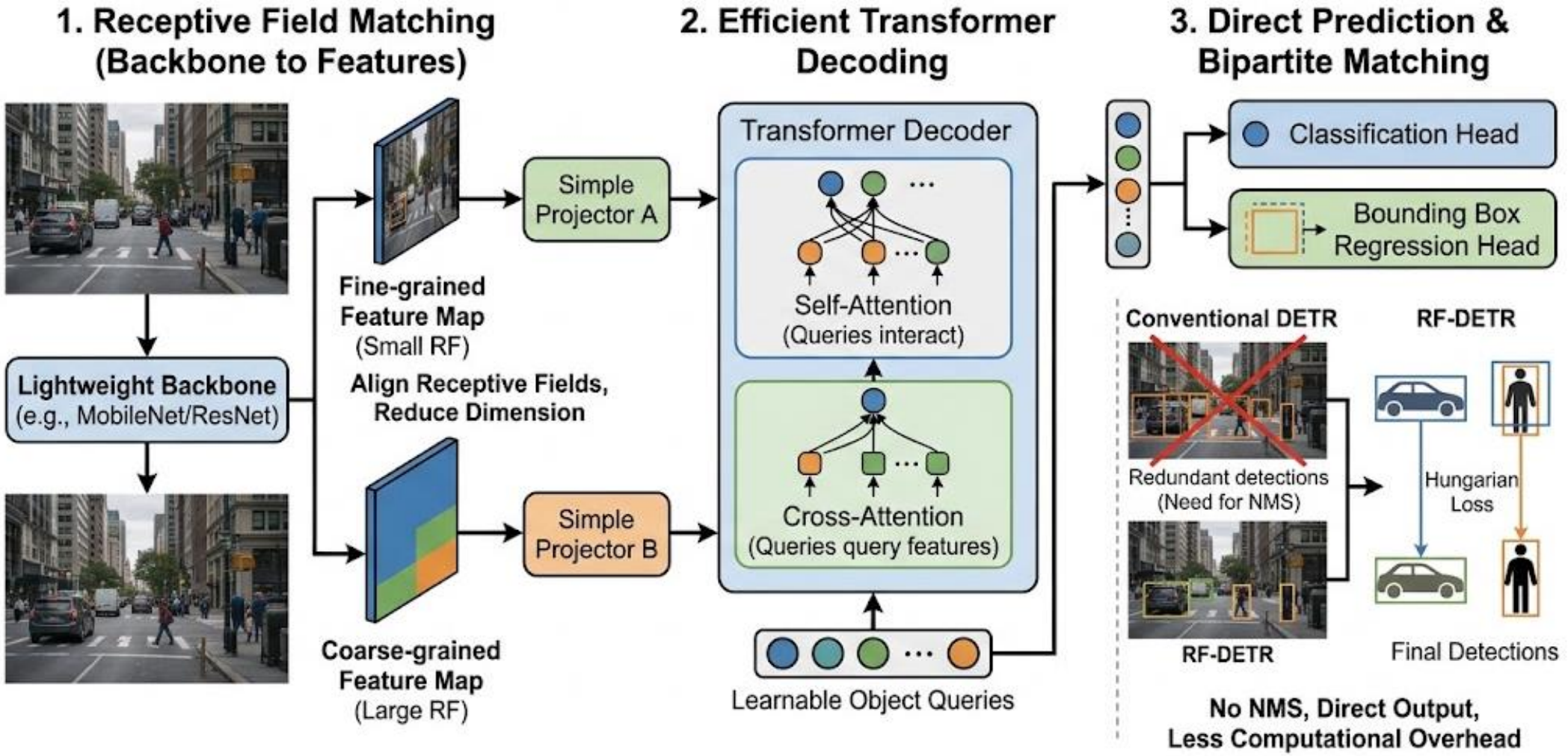


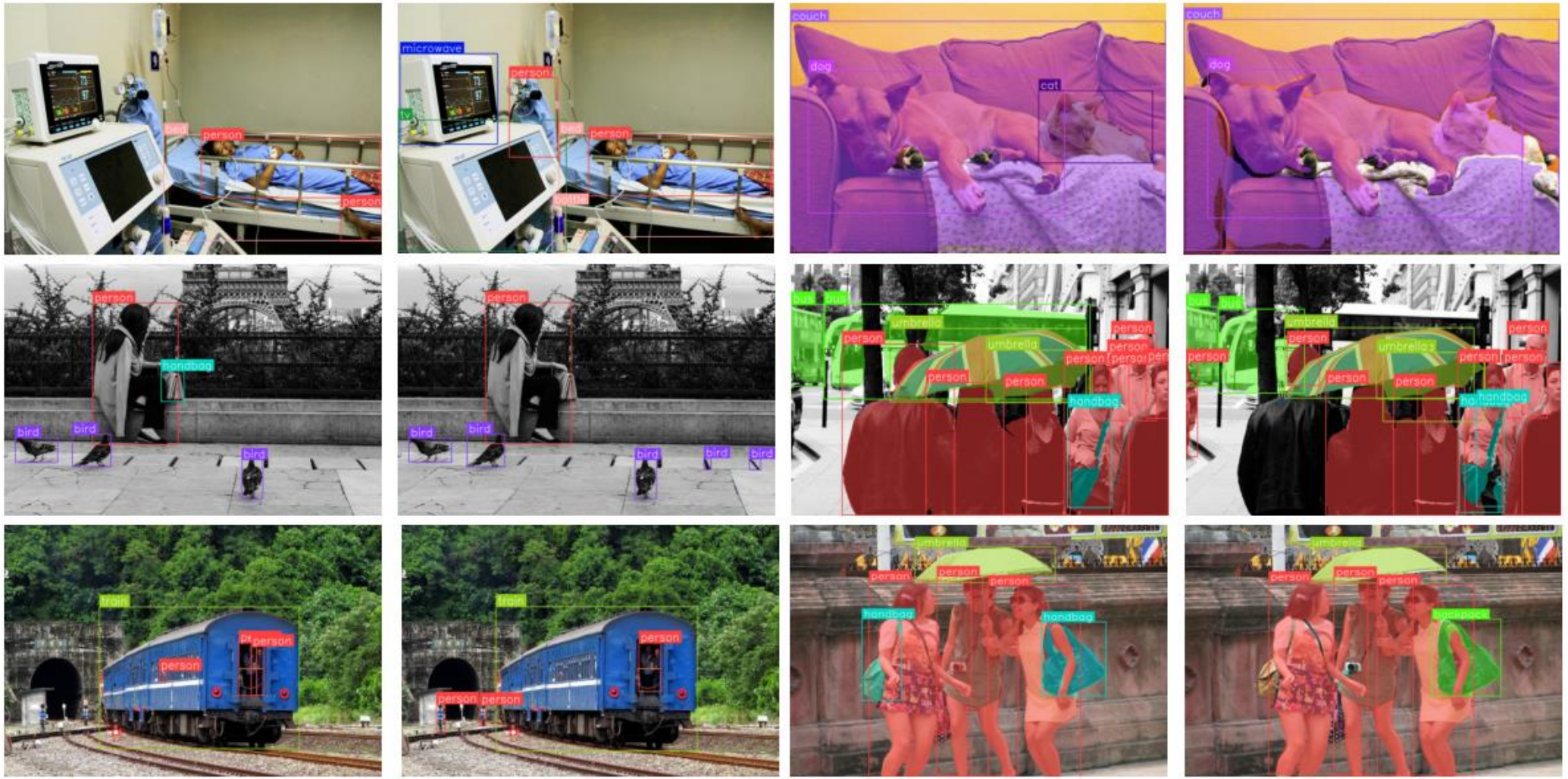
RF-DETR Introduction

- **Receptive Field Matching (Backbone to Features)**: It uses a lightweight backbone and balances multi-scale feature maps by matching receptive fields using simple projectors, capturing both fine details and large contextual areas efficiently.
- **Efficient Transformer Decoding**: Instead of computationally expensive encoder-decoder attention across massive feature maps, query tokens interact directly with projected feature representations via self- and cross-attention mechanisms to locate and classify objects.
- **Bipartite Matching & Direct Prediction**: It uses a Hungarian loss to assign unique ground-truth objects to learnable object queries, directly predicting class labels and bounding box coordinates without needing anchors or Non-Maximum Suppression (NMS).



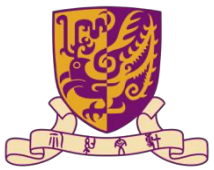
RF-DETR: flow


1. Receptive Field Matching (Backbone to Features)
2. Efficient Transformer Decoding
3. Direct Prediction & Bipartite Matching
Lightweight Backbone (e.g., MobileNet/ResNet)
Fine-grained Feature Map (Small RF)
Coarse-grained Feature Map (Large RF)
Align Receptive Fields, Reduce Dimension
Simple Projector A
Simple Projector B
Transformer Decoder
Self-Attention (Queries interact)
Cross-Attention (Queries query features)
Learnable Object Queries
Classification Head
Bounding Box Regression Head
Conventional DETR
RF-DETR
Redundant detections (Need for NMS)
Hungarian Loss
RF-DETR
Final Detections
No NMS, Direct Output, Less Computational Overhead


RF-DETR (nano)
LW-DETR (tiny)
RF-DETR-Seg (nano)
YOLOv11 (nano)



Feature Domain / Category	YOLO Family	RF-DETR Architecture
Core Backbone	ConvNet (CSPNet / C2f)	Vision Transformer (DINOv2)
Loss Function	CIoU / DFL Box Regression	Hungarian Set-Matching Loss
Post-Processing	Requires NMS Filtering	Native NMS-Free Output
Accuracy Ceiling	Plateaus around 56 mAP	Reaches 60.1 mAP (2XL)
Optimal Hardware Target	CPU Edge Infrastructure: Runs fast on Raspberry Pi and microcontrollers.	GPU Edge Acceleration: Optimized for NVIDIA Jetson and TensorRT.
Use Case / Precision Need	Constrained Memory: Requires ultralight footprints under 10 megabytes.	High Precision Needs: Required for dense occlusion and cluttered scenes.
Training & Fine-Tuning	Scratch Training Speed: Converges in fewer training epochs on custom data.	Out-of-Domain Transfer: Fine-tunes effectively with small dataset sizes.



FaceNet

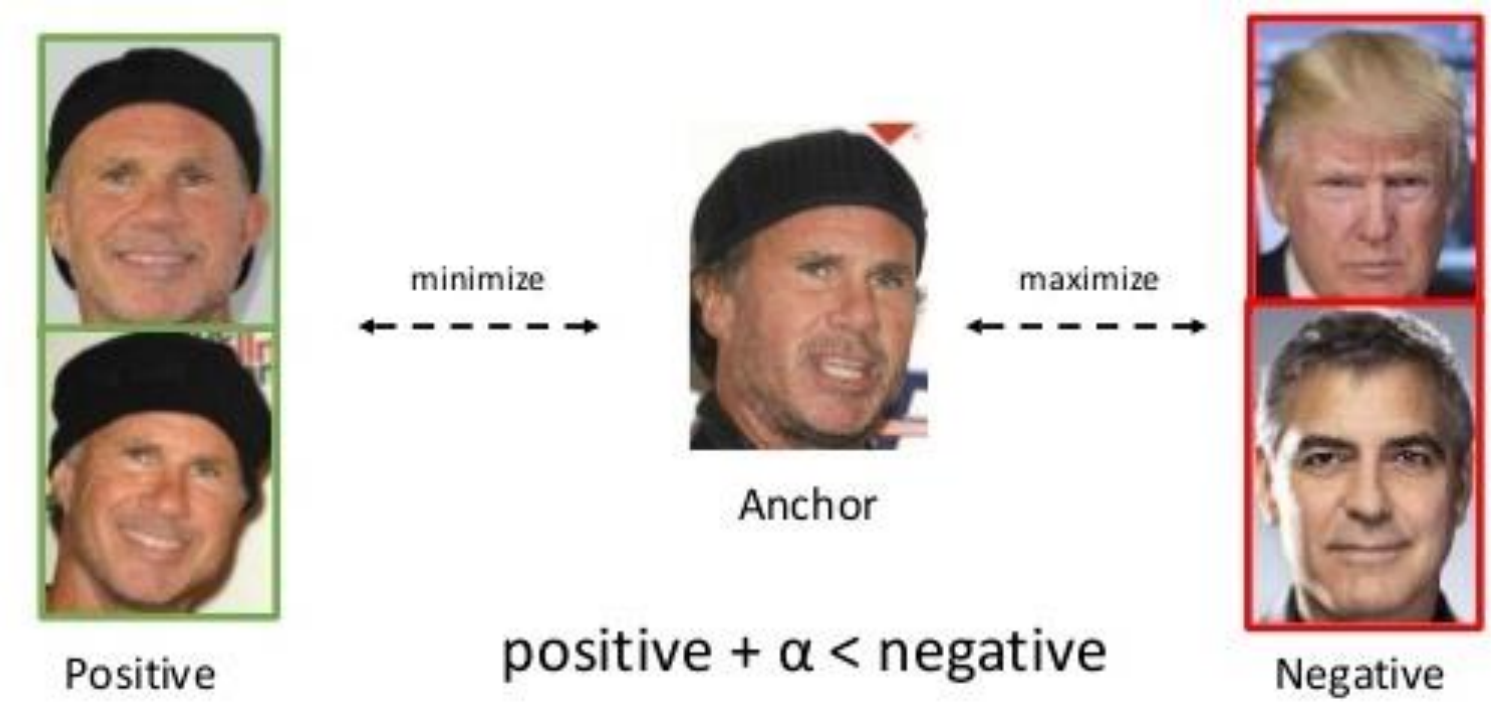
- Map images to a compact Euclidean space, where distances correspond to face similarity



[128-D]

In 128-D space classical techniques can be applied for clustering and categorization, like k-means or KNN

Facenet (Triplet loss)





Recall - Training

How to get the best classification result with least error (the peak)?



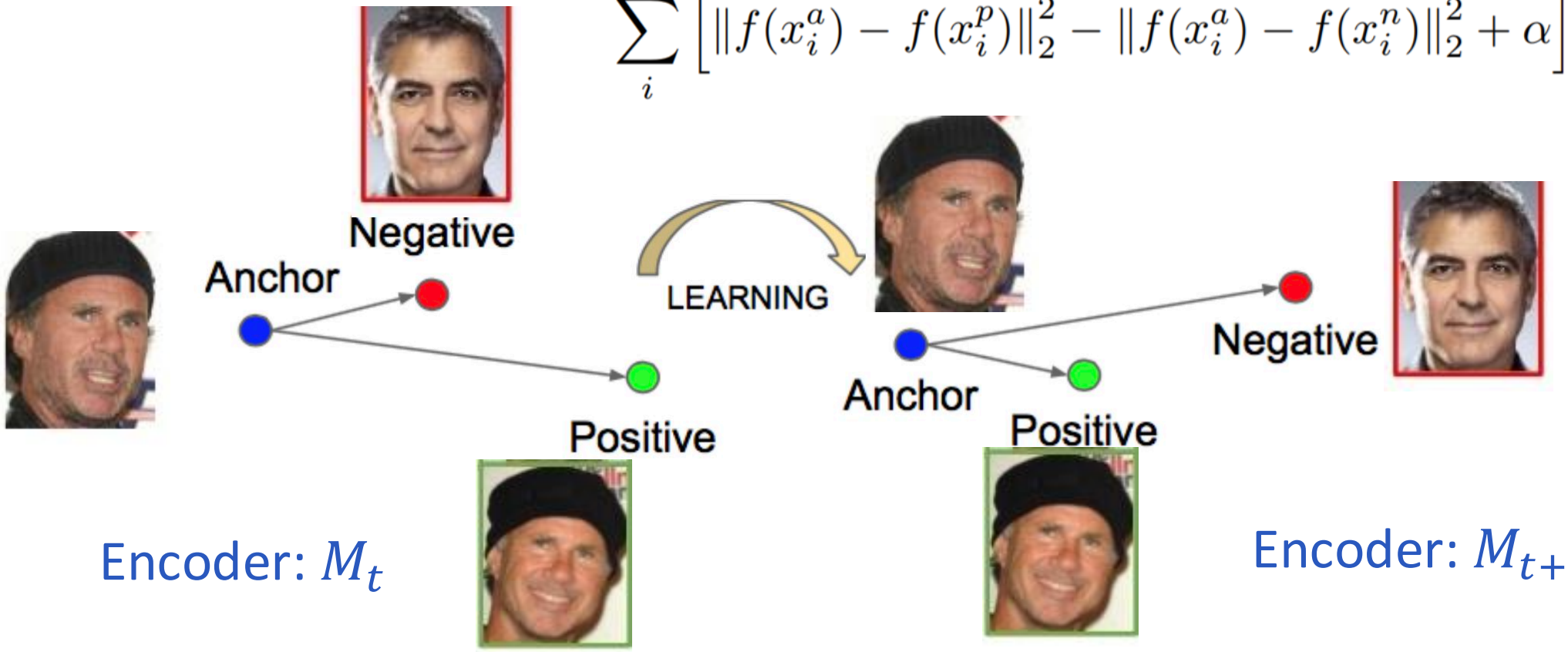
The best m and c are here

You are here



Facenet (Triplet loss)

$$\sum_{i}^{N}\left[\|f(x_i^a)-f(x_i^p)\|_2^2-\|f(x_i^a)-f(x_i^n)\|_2^2+\alpha\right]$$

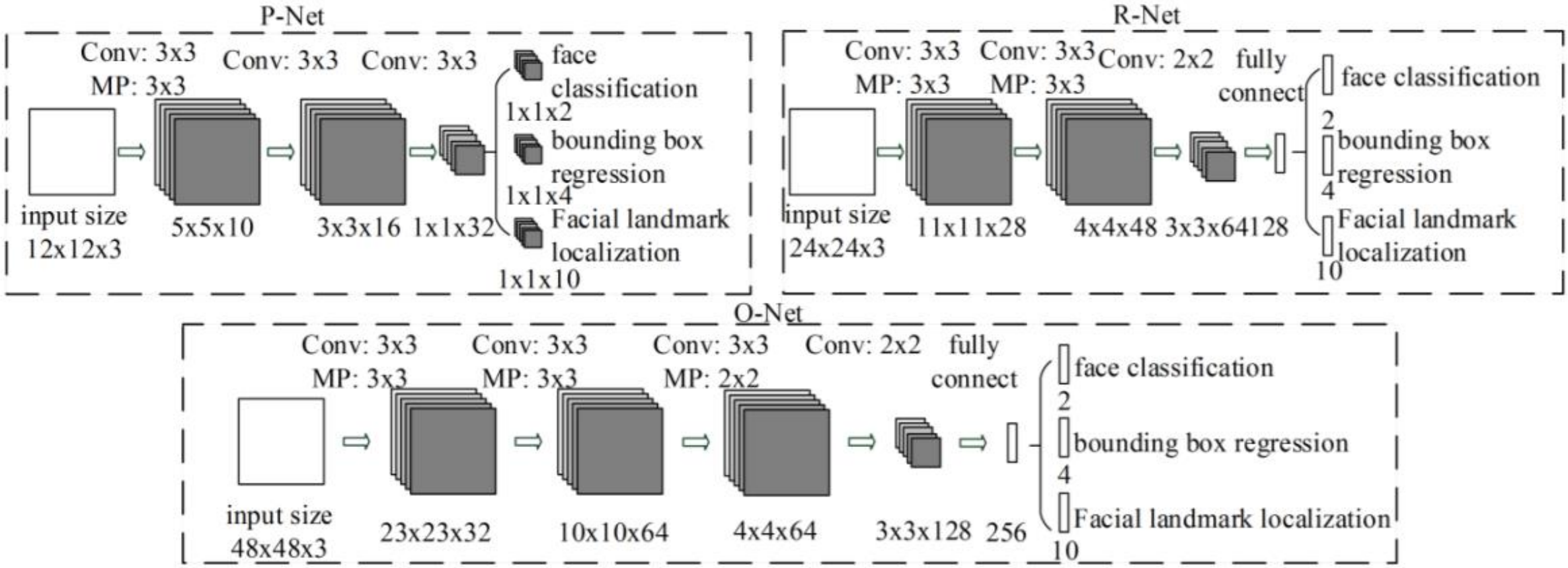


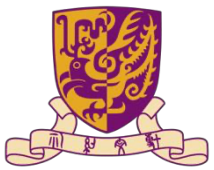
Encoder: M_t

Encoder: M_{t+1}

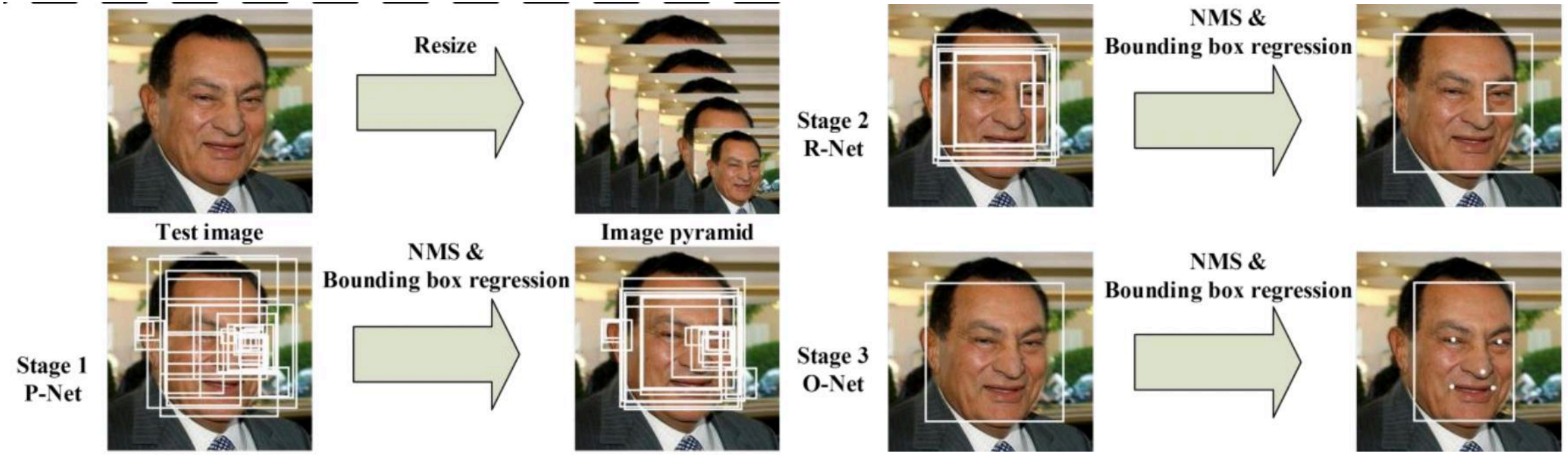
Questions: why not using classification?

MT-CNN for Face Detection





MT-CNN for Face Detection



MT-CNN and Facenet

```python
# Load pre-trained models
mtcnn = MTCNN()
facenet_model = load_model('facenet_model.h5')

# Function to detect and encode faces
def detect_encode_faces(image_path):
    image = cv2.imread(image_path)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    faces = mtcnn.detect_faces(image)
    face_embeddings = []
    face_img = []

    for face in faces:
        bounding_box = face['box']
        x1, y1, width, height = bounding_box
        x2, y2 = x1 + width, y1 + height

        face_img = image[y1:y2, x1:x2]
        face_img = cv2.resize(face_img, (160, 160))
        face_img = img_to_array(face_img)
        face_img = preprocess_input(face_img)
        face_img = np.expand_dims(face_img, axis=0)

        embedding = facenet_model.predict(face_img)[0]
        face_embeddings.append(embedding)
        face_img.append(face_img)

    return face_embeddings, face_img
```





Face Clustering with FaceNet

```python
# Function for face clustering using DBSCAN
def cluster_faces(face_embeddings, face_img, eps=0.5, min_samples=5):
    X = np.array(face_embeddings)
    clustering = DBSCAN(eps=eps, min_samples=min_samples).fit(X)
    labels = clustering.labels_

    unique_labels = np.unique(labels)
    clusters = []

    for label in unique_labels:
        cluster = []
        indices = np.where(labels == label)[0]
        for idx in indices:
            cluster.append(face_img[idx])
        clusters.append(cluster)

    return clusters
```



Human Joints Detection

```
model_name = "movenet_lightning" #@param ["movenet_lightning", "movenet_thunder", "
input_size = 192
# module = hub.load("https://tfhub.dev/google/movenet/singlepose/lightning/4")


# Initialize the TFLite interpreter
interpreter = tf.lite.Interpreter(model_path="model.tflite")
interpreter.allocate_tensors()

def movenet(input_image):
  input_image = tf.cast(input_image, dtype=tf.uint8)
  input_details = interpreter.get_input_details()
  output_details = interpreter.get_output_details()
  interpreter.set_tensor(input_details[0]['index'], input_image.numpy())
  # Invoke inference.
  interpreter.invoke()
  # Get the model prediction.
  keypoints_with_scores = interpreter.get_tensor(output_details[0]['index'])
  return keypoints_with_scores
```



Demo: OCR

```python
import easyocr
import cv2
import matplotlib.pyplot as plt

# Initialize the EasyOCR reader
reader = easyocr.Reader(['en'])  # Specify the languages you want to recognize

# Load an image using OpenCV
image_path = 'path_to_your_image.jpg'
image = cv2.imread(image_path)

# Perform OCR on the image
results = reader.readtext(image)

# Print results
for (bbox, text, prob) in results:
    print(f"Detected text: {text} with confidence: {prob:.2f}")

    # Draw bounding box around the detected text
    (top_left, top_right, bottom_right, bottom_left) = bbox
    top_left = tuple(map(int, top_left))
    bottom_right = tuple(map(int, bottom_right))

    # Draw the rectangle on the image
    cv2.rectangle(image, top_left, bottom_right, (0, 255, 0), 2)

# Display the image with detected text
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.axis('off')  # Hide axes
plt.show()
```



Libraries

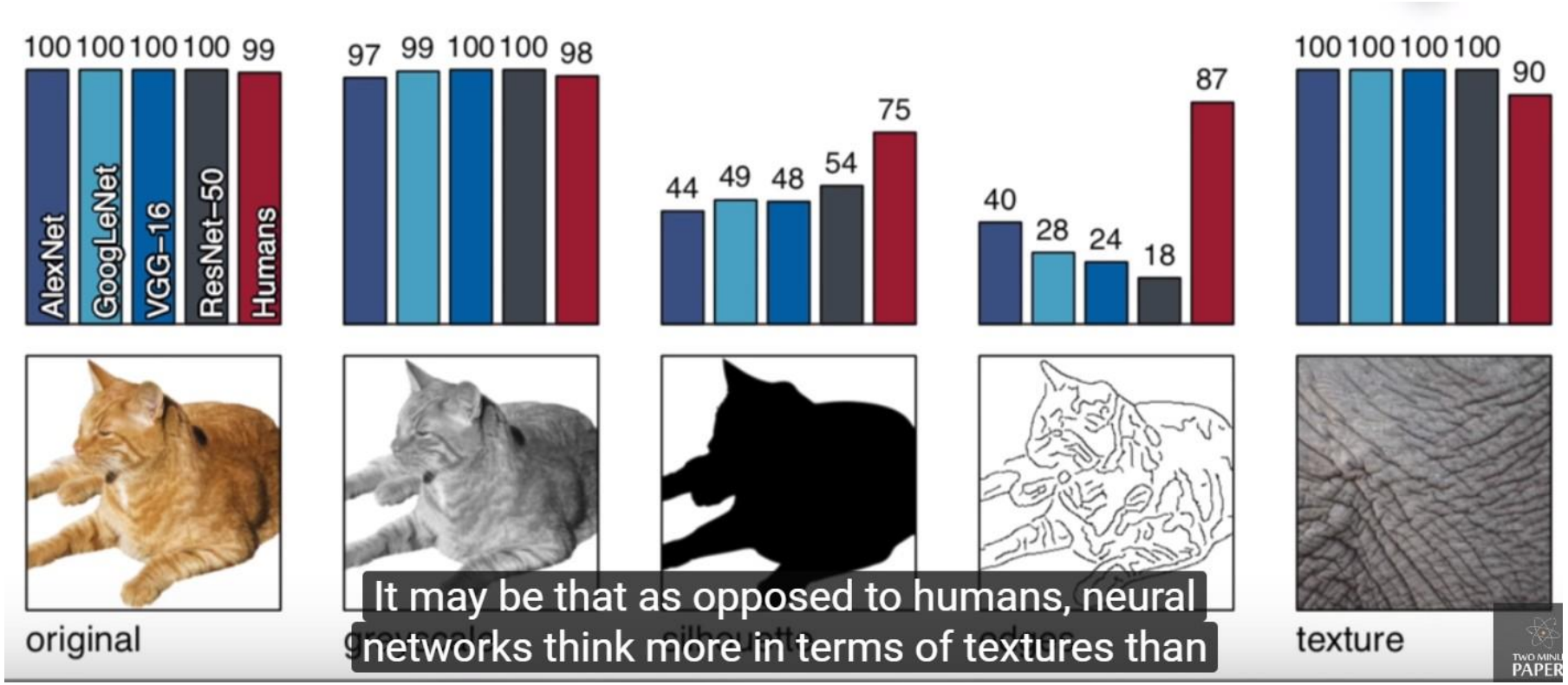
Model Zoo

- https://github.com/tensorflow/models/blob/63ecef1a3513b00c01f6aed75e178636746eff71/research/object_detection/g3doc/detection_model_zoo.md



CNNs do NOT Think Like Humans

- https://www.youtube.com/watch?v=YFL-MI5xzgg&t=105s



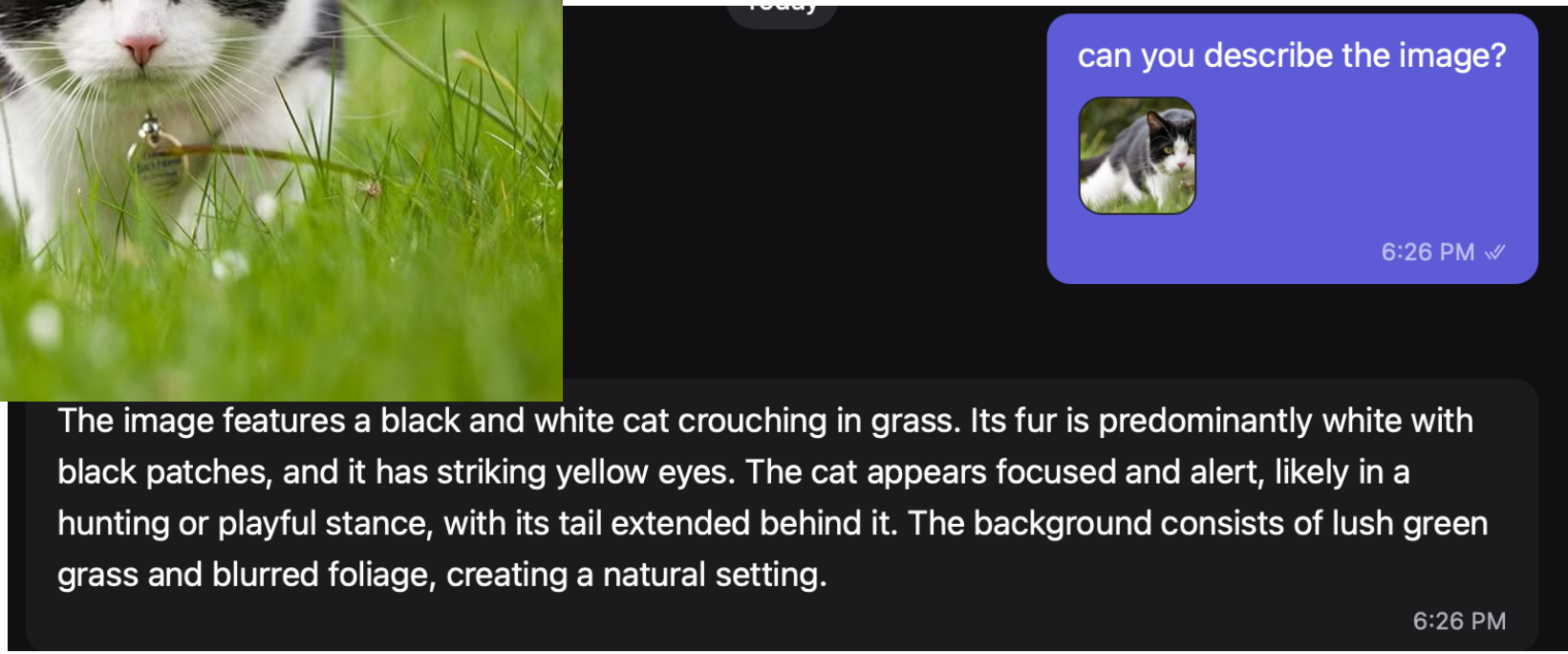
Agenda

- Computer Vision (CV) and Convolutional Neural Networks (CNN)
- Implementation of CV (OpenCV+PyTorch)
- Multi-model LLM and VLM



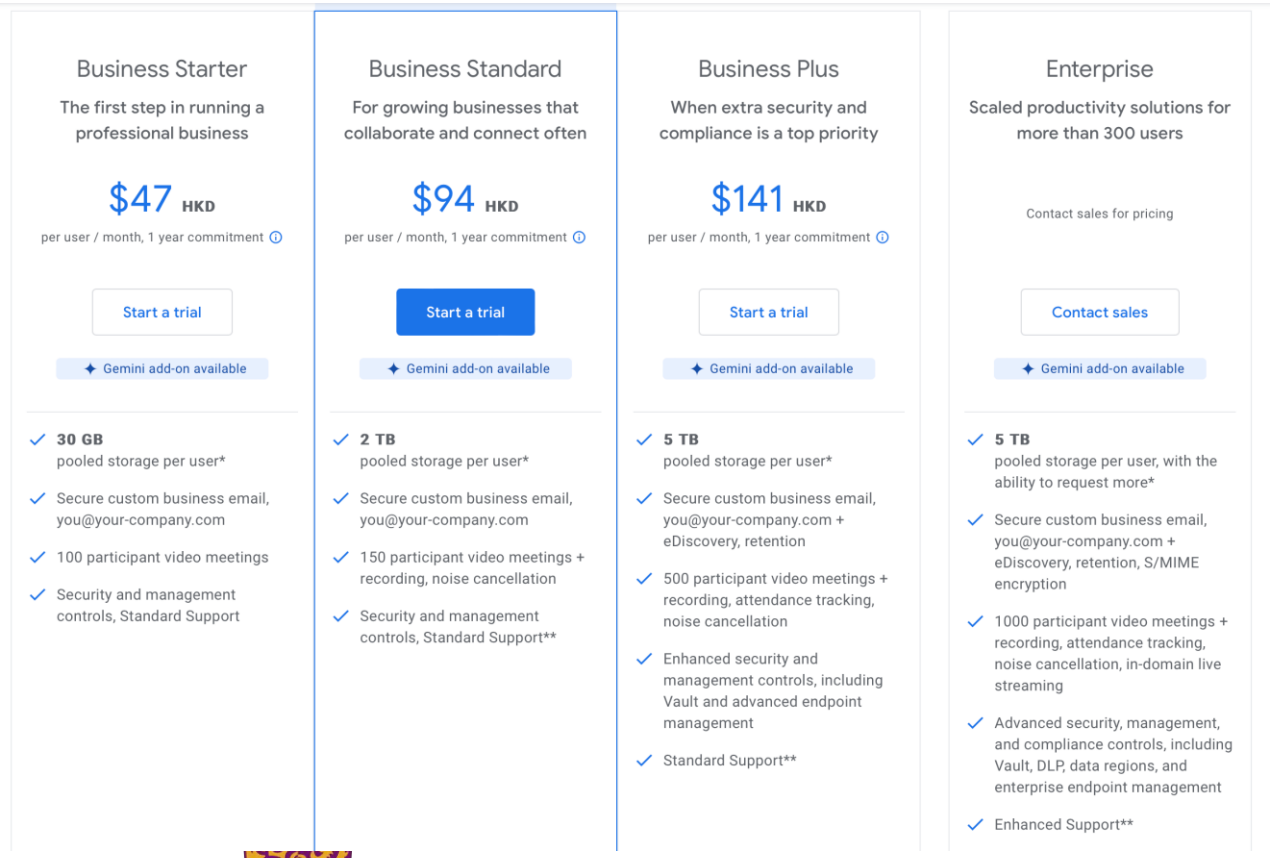
Understand images





Understanding tables with texts

I am a startup with 10 people. Which plan is good for me and why



Business Starter	Business Standard	Business Plus	Enterprise
The first step in running a professional business	For growing businesses that collaborate and connect often	When extra security and compliance is a top priority	Scaled productivity solutions for more than 300 users
$47 HKD per user / month, 1 year commitment	$94 HKD per user / month, 1 year commitment	$141 HKD per user / month, 1 year commitment	Contact sales for pricing
Start a trial	Start a trial	Start a trial	Contact sales
Gemini add-on available	Gemini add-on available	Gemini add-on available	Gemini add-on available
30 GB pooled storage per user*	**2 TB** pooled storage per user*	**5 TB** pooled storage per user*	**5 TB** pooled storage per user, with the ability to request more*
Secure custom business email, you@your-company.com	Secure custom business email, you@your-company.com	Secure custom business email, you@your-company.com + eDiscovery, retention	Secure custom business email, you@your-company.com + eDiscovery, retention, S/MIME encryption
100 participant video meetings	150 participant video meetings + recording, noise cancellation	500 participant video meetings + recording, attendance tracking, noise cancellation	1000 participant video meetings + recording, attendance tracking, noise cancellation, in-domain live streaming
Security and management controls, Standard Support	Security and management controls, Standard Support**	Enhanced security and management controls, including Vault and advanced endpoint management	Advanced security, management, and compliance controls, including Vault, DLP, data regions, and enterprise endpoint management
		Standard Support**	Enhanced Support**

Choosing the right plan for your startup with 10 people depends on your specific needs regarding storage, collaboration, and security. Here's a breakdown of the options:

1. Business Starter ($47 HKD/user/month)

- **Storage**: 30 GB pooled per user
- **Video Meetings**: 100 participants
- **Best For**: Very small teams or those with minimal storage and collaboration needs.
- **Pros**: Most cost-effective for basic needs.
- **Cons**: Limited storage and features for larger teams.

2. Business Standard ($94 HKD/user/month)

- **Storage**: 2 TB pooled per user
- **Video Meetings**: 150 participants
- **Best For**: Growing businesses needing more collaboration tools.
- **Pros**: Good storage and meeting capacity for a team of your size.
- **Cons**: Higher cost than the starter plan.

3. Business Plus ($141 HKD/user/month)

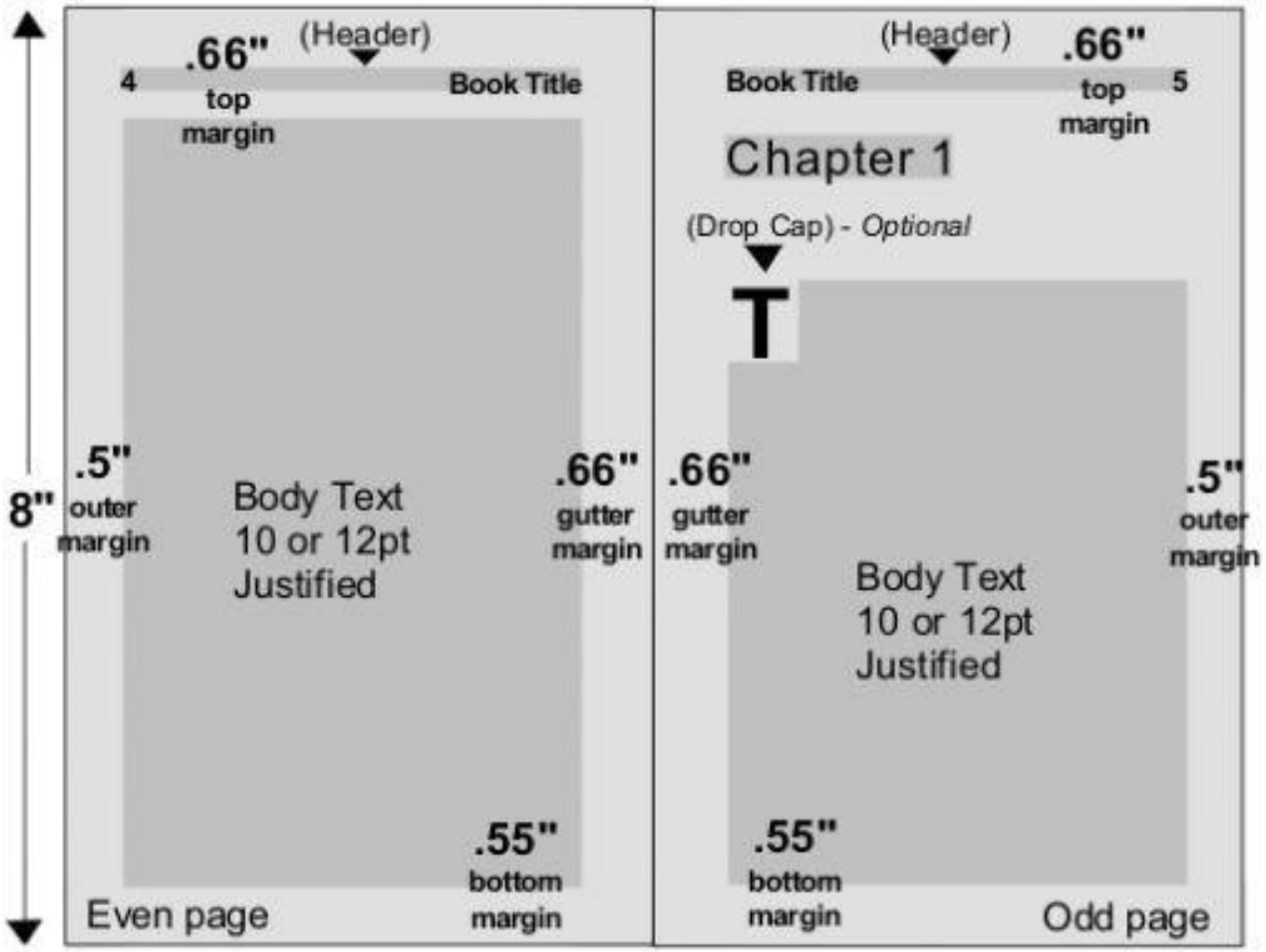
- **Storage**: 5 TB pooled per user
- **Video Meetings**: 500 participants
- **Best For**: Teams focused on security and compliance.
- **Pros**: Enhanced features such as eDiscovery, which could be beneficial if you handle sensitive information.
- **Cons**: More expensive; may offer more than you need at this stage.

Recommendation:

Business Standard seems to be the best fit for your startup. It balances cost and features well, providing ample storage and meeting capacity without the higher cost of the Business Plus plan. This plan supports collaboration and growth as your team expands.

If your business involves handling sensitive information or requires advanced compliance features, consider the Business Plus plan, but it may be more than necessary right now.

Understand difficult image



How many words in 1 page?

GPT-4o-Mini Poe ···

To estimate the number of words on a page based on the provided layout, we can follow these steps:

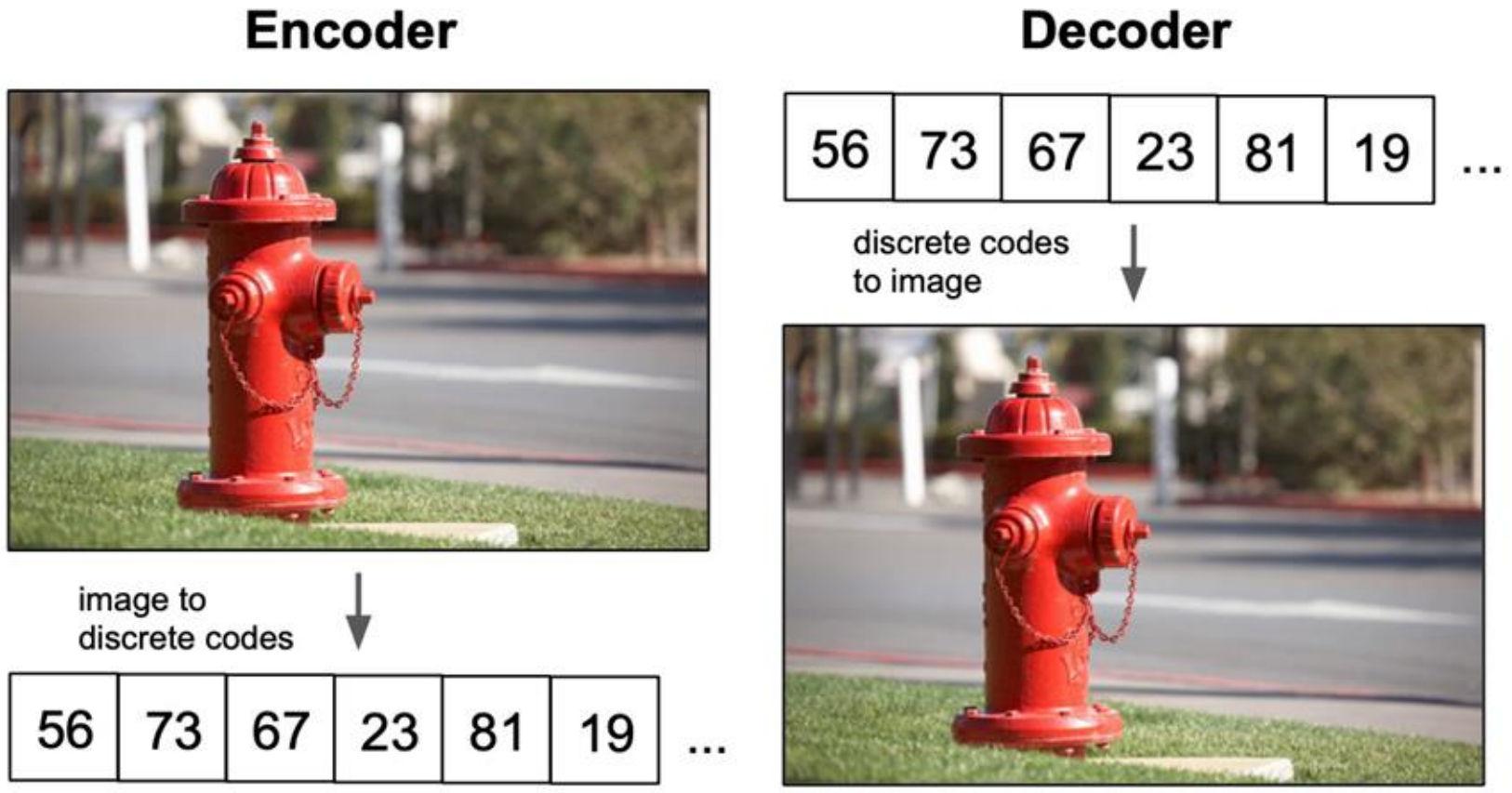
1. **Page Dimensions**: The page is 8 inches tall and 4 inches wide (for each side of the spread).
2. **Margins**:
   - Top margin: 0.66 inches
   - Bottom margin: 0.55 inches
   - Outer margins: 0.5 inches
   - Gutter margin: 0.66 inches
3. **Usable Space**:
   - Height: $8 - 0.66 - 0.55 = 6.79$ inches
   - Width for one side: $4 - 0.5 - 0.5 = 3$ inches
4. **Text Area**:
   - Height: 6.79 inches
   - Width: 3 inches
5. **Font Size and Words per Line**:
   - Assuming a 12-point font (approximately 1/6 inch per line), you can fit about $\frac{6.79}{\frac{1}{6}} \approx 40.74$ lines (round down to 40).
   - Average word length (including spaces) is around 5-6 characters. If we assume an average of 10-12 words per line (considering spacing), you can fit approximately 400-480 words per page.

Conclusion

Estimated Word Count: Approximately **400 to 480 words** per page, depending on formatting and spacing.

Vector Quantized-VAE (VQVAE)

- VAE: learns a continuous latent representation
- Vector Quantized-VAE: learns a discrete latent representation



ref: Neural Discrete Representation Learning

Learning Strategies

- Contrastive Learning
- PrefixLM
- Multi-modal Fusing with Cross Attention
- MLM / ITM
- No Training



ref: https://huggingface.co/blog/vision_language_pretraining

Contrastive Learning: Multimodal Representations

- Multimodality is the scientific study of heterogeneous (e.g., texts, images, audios) and interconnected data

- Goal: to represent and summarize multimodal data in joint representations s.t. complementary information is preserved

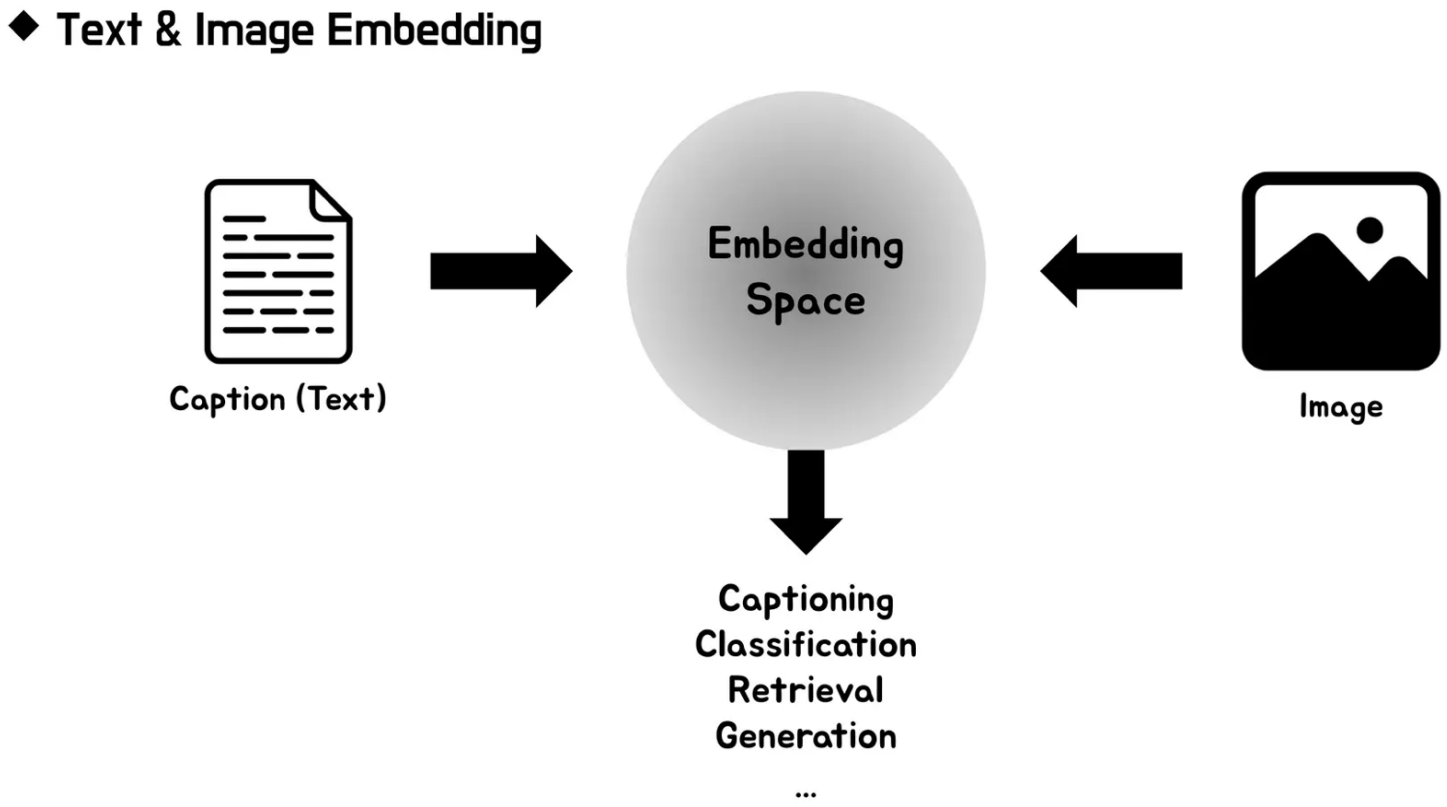
- redundant parts of the modalities are filtered out



- This presentation will only focus on Vision-Language Pre-training (VLP)

Contrastive Language-Image Pre-Training (CLIP)

- During training
  - batches of (x, c) pairs are sampled from a large dataset, and the model optimizes a contrastive cross-entropy loss
  - encourages a high dot-product f(x) · g(c), for a pair x and c



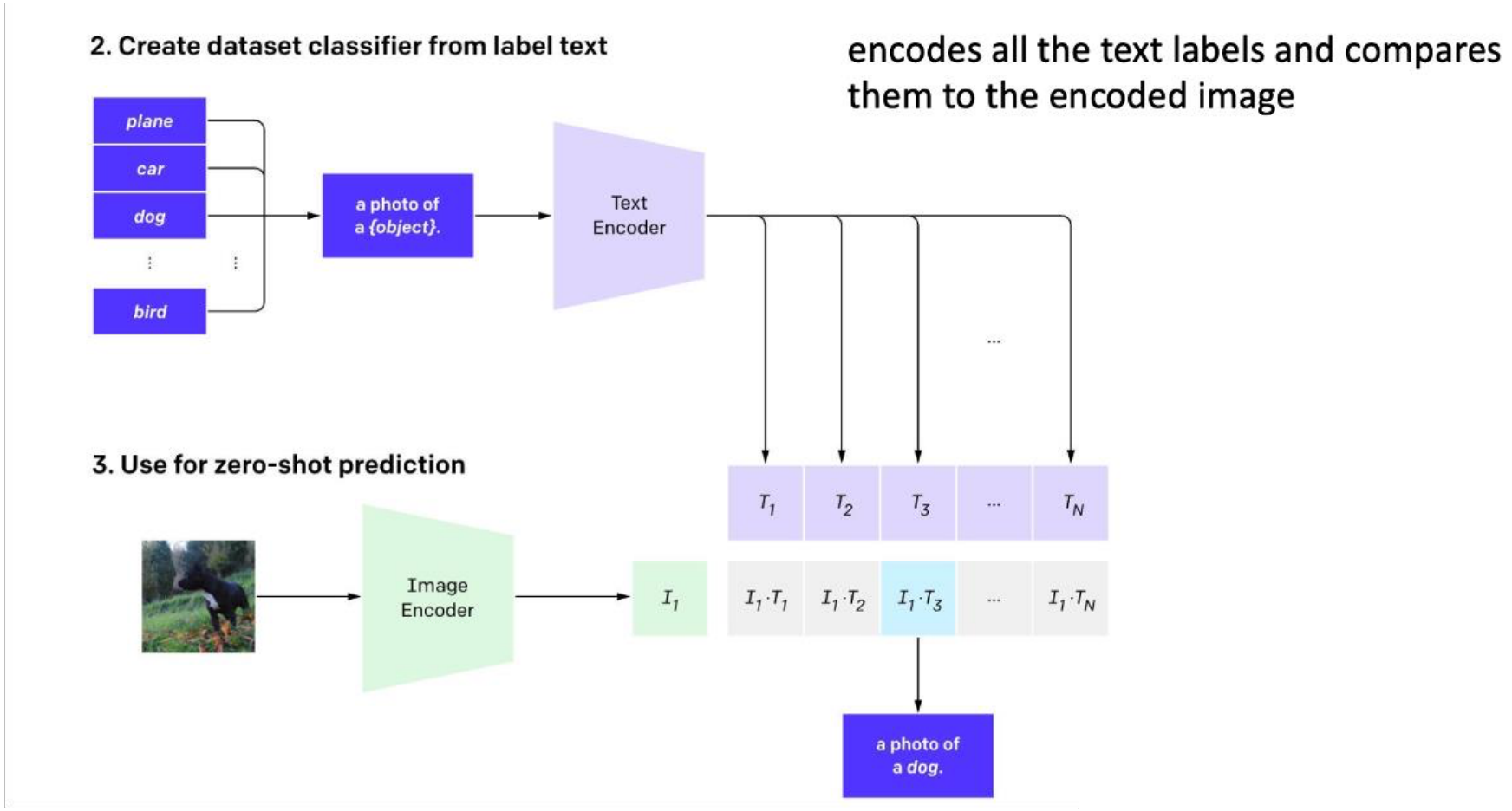
Vision-language models: Contrastive learning

- Making embedding of (image, text) pairs similar and that of non-pairs dissimilar
- This embedding space is super helpful for performing searches across modalities
- Can return the best caption given an image
- Has impressive capabilities for zero-shot adaptation to unseen tasks, without the need for fine-tuning



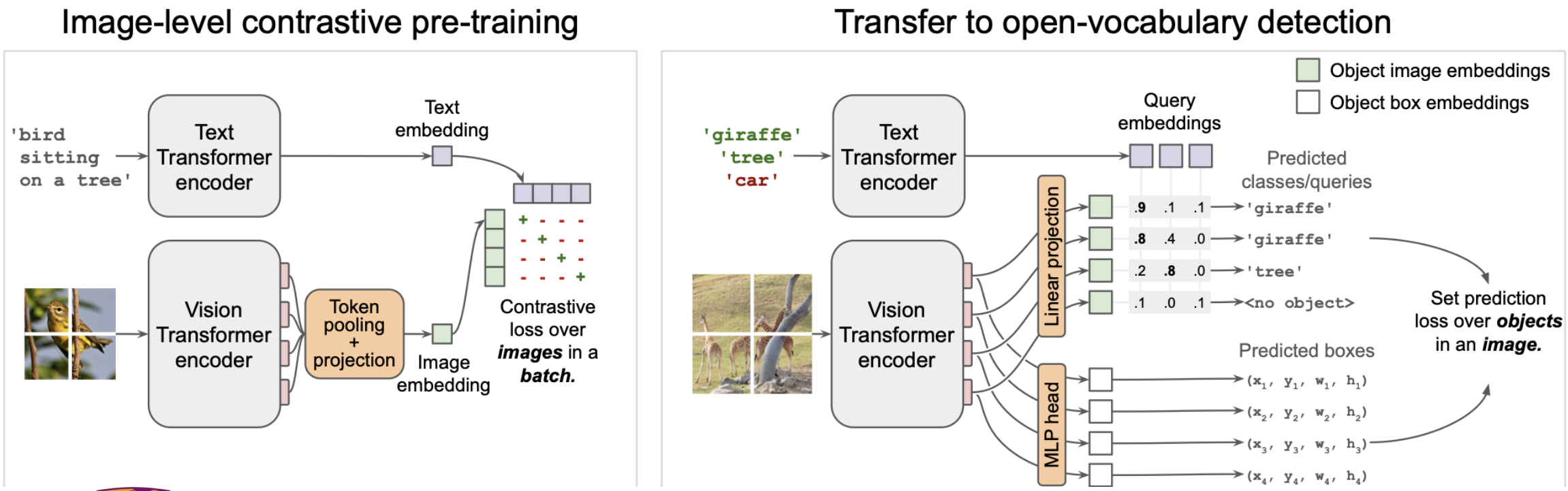
Ref:Radford et al. "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021

CLIP for zero-shot learning



Extending CLIP

- Available: https://huggingface.co/google/owlvit-base-patch32
- Find the location and the label of objects on an image



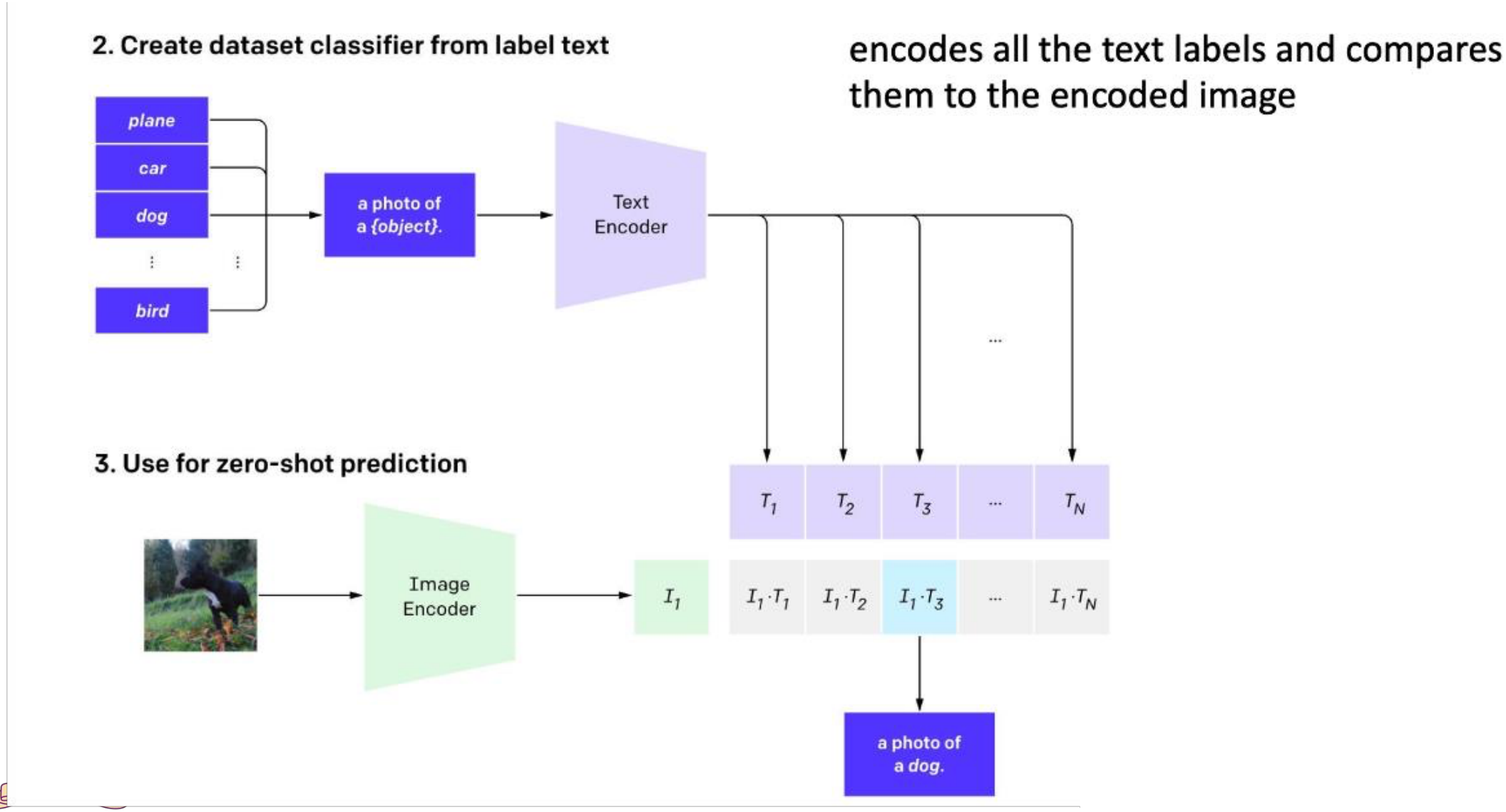
ref: Simple Open-Vocabulary Object Detection with Vision Transformers

CLIP: Summary

- CLIP improved open-vocabulary visual recognition capabilities through learning from Internet-scale image-text pairs.
- CLIP doesn’t go directly from image to text or vice versa. It just connecting the image and text embedding spaces
- CLIP can only address limited use cases such as classification
  - It crucially lack the ability to generate language which makes them less suitable
- to more open-ended tasks such as captioning or visual question answering



Demo: using CLIP

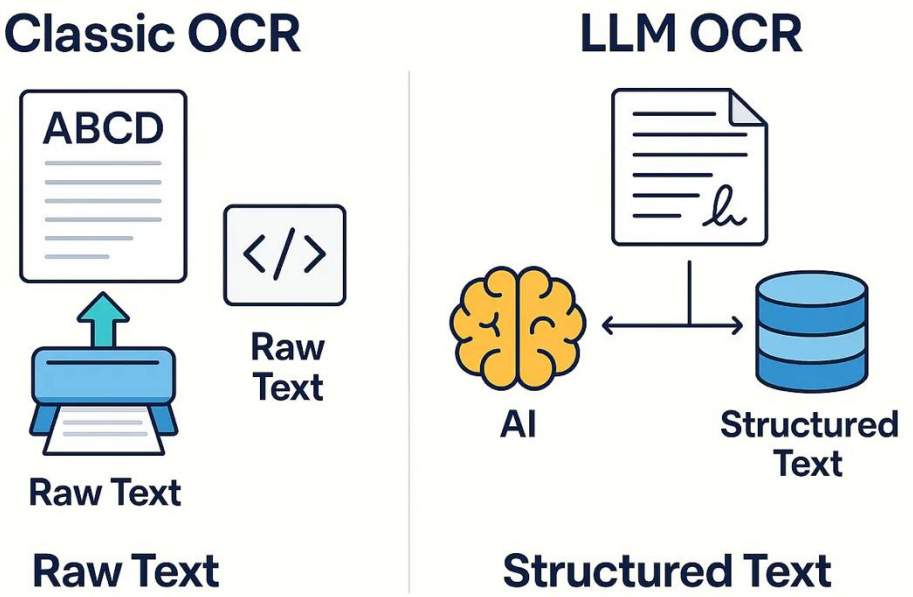


Demo: OCR and VLM (Gemma 4 E2B, through Ollama)

```python
client = openai.OpenAI(
    api_key=key_poe,  # or os.getenv("POE_API_KEY")
    base_url="https://api.poe.com/v1",
)

prompt = "You are a data entry and you need to extract the text from image."

# Path to your image file
image_path = f"{data_path}/{target_file}.jpg"  # Replace with actual image path
image_base64 = encode_image(image_path)
chat = client.chat.completions.create(
    model="Gemini-2.5-Flash",
    messages=[
        {
            "role": "user",
            "content": [
```



Sample Short Questions

- Given 3 vectors: (1,2), (4,-1) and (11,-9), compute the distances:
  - Euclidean
  - Manhattan
  - Cosine

Euclidean Distance

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Straight line distance

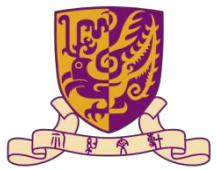
Manhattan Distance

$$d = |x_1 - x_2| + |y_1 - y_2|$$

Grid-like distance

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

- Regarding to transfer learning:
  - what are the advantages of transfer learning?
  - what are the limitations of transfer learning?
  - (answer in 3 sentences each)



Sample MC questions (use LLM to generate more for practice)

- What primary training objective does CLIP use to align visual and text representations?
  - A) Generative adversarial learning
  - B) Contrastive loss across image-text pairs
  - C) Masked language modeling (MLM)
  - D) Next-token prediction
- Why are positional encodings necessary in the vanilla Transformer architecture?
  - A) To prevent overfitting on short sequence lengths.
  - B) To help the model converge faster during gradient descent.
  - C) To reduce the dimensional footprint of input embeddings.
  - D) Because self-attention is naturally permutation-invariant (no order).



Questions?

Email: pakmingcheung@cuhk.edu.hk

